

ÉCRIRE UN ENTIER POSITIF EN BINAIRE

1) Compter en binaire

Historiquement, on doit la naissance des premiers ordinateurs à l'association de trois idées géniales :

- 1) Créer une machine à calculer qui soit électrique et non mécanique
- 2) Calculer en binaire : 0 = pas de courant, 1 = courant (cf travaux de Leibnitz vers 1700)
- 3) Utiliser l'algèbre de Boole (cf travaux de Georges Boole vers 1850)

Les ordinateurs actuels étant toujours basé sur ces 3 idées, il nous faut donc notamment apprendre à compter en binaire !

décimal (base 10)	binaire (base 2)	hexadécimal (base 16)
0	0	0
1	1	1
2	10	2
3	11	3
4		4
5		5
6		6
7		7
8		8
9		9
10		A
11		B
12		C
13		D
14		E
15		F
16		
17		

Remarque :

Les deux chiffres binaires 0 et 1 sont appelés en anglais « binary digit » qui a été contracté en « bit ».

II) Conversions décimal $\Leftrightarrow$ binaire

1) Premières puissances de 2 :

2^n	2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7
valeur	1	2	4	8				
binaire	1	10	100					

2) Décimal vers binaire :

a) *Méthode 1 : En décomposant le nombre décimal en somme de puissances de 2 :*

$$\begin{aligned}
 \text{Ex : } 173 &= 128 + 32 + 8 + 4 + 1 \\
 &= 2^7 + 2^5 + 2^3 + 2^2 + 2^0 \\
 &= \boxed{}
 \end{aligned}$$

b) *Méthode 2 : En effectuant des divisions entières par 2 :*

$$\text{Ex : } 173 = 2 \times 86 + \mathbf{1}$$

$$86 = 2 \times 43 + \mathbf{0}$$

$$43 = 2 \times 21 + \mathbf{1}$$

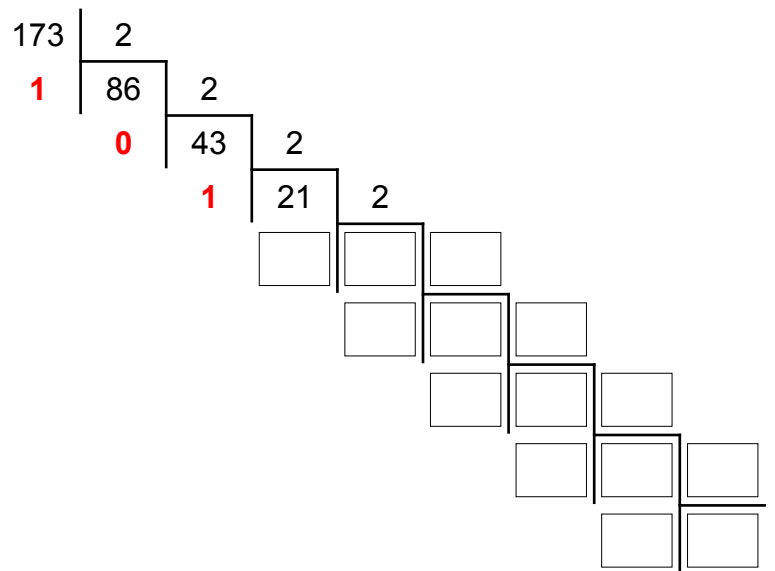
$$\boxed{} = 2 \times \boxed{} + \boxed{}$$

$$\boxed{} = 2 \times \boxed{} + \boxed{}$$

$$\boxed{} = 2 \times \boxed{} + \boxed{}$$

$$\boxed{} = 2 \times \boxed{} + \boxed{}$$

$$\boxed{} = 2 \times \boxed{} + \boxed{}$$



En lisant les restes de bas en haut, on retrouve le résultat : **10101101**₂

Explication :

Refaisons exactement les mêmes calculs que ci-dessus mais directement en base 2 :

$$10101101 = 10 \times 1010110 + \mathbf{1}$$

$$1010110 = 10 \times 101011 + \mathbf{0}$$

$$101011 = 10 \times 10101 + \mathbf{1}$$

$$10101 = 10 \times 1010 + \mathbf{1}$$

$$1010 = 10 \times 101 + \mathbf{0}$$

$$101 = 10 \times 10 + \mathbf{1}$$

$$10 = 10 \times 1 + \mathbf{0}$$

$$1 = 10 \times 0 + \mathbf{1}$$

3) Binaire vers décimal :

$$\begin{aligned}\text{Ex : } 1101001_2 &= 2^0 + \boxed{} \\ &= 1 + \boxed{} \\ &= \boxed{}\end{aligned}$$

III) Conversions binaire $\Leftrightarrow$ hexadécimal

Puisque $1111_2 = F_{16}$, il suffit de regrouper les bits par groupes de 4 !

$$\text{Ex : } 0110\ 1001_2 = 69_{16}$$

$$7F_{16} = 0111\ 1111_2$$

IV) Conversions décimal $\Leftrightarrow$ hexadécimal

On utilise le même principe que pour les conversions de décimal vers binaire sauf que l'on utilise les puissances de 16 et non celles de 2.

16^0	16^1	16^2	16^3
1	16	256	4096

$$\begin{aligned}\text{Ex : } 723 &= \boxed{} \times 16^2 + \boxed{} \times 16 + \boxed{} \\ &= \boxed{}\end{aligned}$$

$$\begin{array}{r|l} \text{Ex : } 723 & 16 \\ \hline \boxed{} & \boxed{} \boxed{} \\ \boxed{} & \boxed{} \boxed{} \boxed{} \\ \boxed{} & \boxed{} \boxed{} \end{array}$$

$$\begin{aligned}\text{Ex : } A1F_{16} &= \boxed{} \times 16^2 + \boxed{} \times 16 + \boxed{} \\ &= \boxed{}\end{aligned}$$

V) Opérations posées en binaire :

On peut faire des opérations « posées » en binaire en utilisant les méthodes apprises en primaire pour le système décimal :

Exemple d'addition :

$$\begin{array}{r} 1\ 1\ 0\ 1\ 1\ 0 \\ + \quad\quad 1\ 0\ 1\ 1 \\ \hline \square\square\square\square\square\square\square \end{array}$$

Exemple de multiplication

$$\begin{array}{r} 1\ 1\ 0\ 1\ 1\ 0 \\ \times \quad\quad 1\ 0\ 1\ 1 \\ \hline \square\square\square\square\square\square\square\square\square\square \\ \square\square\square\square\square\square\square\square\square\square \\ \square\square\square\square\square\square\square\square\square\square \\ \hline \square\square\square\square\square\square\square\square\square\square \end{array}$$

VI) Quels entiers positifs peut-on coder avec n bits ?

$1_2 = 1$ donc avec 1 bit, on peut coder les entiers positifs de 0 à

$11_2 = 3$ donc avec 2 bits, on peut coder les entiers positifs de 0 à

$111_2 = 7$ donc avec 3 bits, on peut coder les entiers positifs de 0 à

donc avec n bits, on peut coder les entiers positifs de 0 à

VII) Avec Python

>>> 0b11

>>> 0x1F

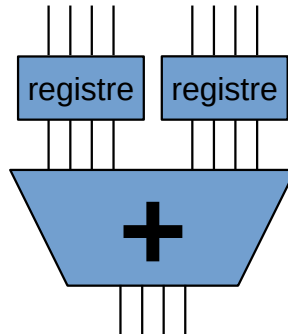
>>> bin(31)

>>> hex(31)

CODER UN ENTIER « SIGNÉ » EN BINAIRE

1) Opposé d'un entier en binaire

Le premier microprocesseur commercialisé par Intel, le 4004, avait une architecture 4 bits. Cela veut notamment dire qu'il était « câblé » pour faire directement des opérations avec des entiers codés sur 4 bits. Il pouvait par exemple additionner directement deux entiers codés sur 4 bits en un seul cycle d'horloge et fournir un résultat également codé sur 4 bits :



Bien sûr, dès que les fabricants en ont été capables, ils se sont mis à construire des processeurs 8 bits, puis 16 bits, 32 bits et aujourd'hui 64 bits.

Ainsi donc, pour un processeur donné, les entiers sont toujours codés avec un nombre fixe de bits. Du coup, pour représenter l'opposé d'un entier, les fabricants ont eu l'idée d'utiliser la technique du « complément à 2^n ».

Principe dans le cas d'un processeur 8 bits :

Cherchons par exemple l'opposé du nombre de 8 bits : $A = 0010\ 1001$

On inverse les bits (Complément à 1 de chaque bit) : $B =$

On remarque que, quelle que soit la valeur de A, on aura : $A+B =$

Si on ajoute 1, on obtiendra donc toujours : $A+B+1 =$

Comme on travaille sur 8 bits, on supprime le 9ème bit : $A+B+1 =$

Bilan :

Quelle que soit la valeur de A, le nombre B+1 est ce qu'il faut ajouter à A pour obtenir $1\ 0000\ 0000_2 = 2^8$. On appellera donc ce nombre « complément à 2^8 » de A.

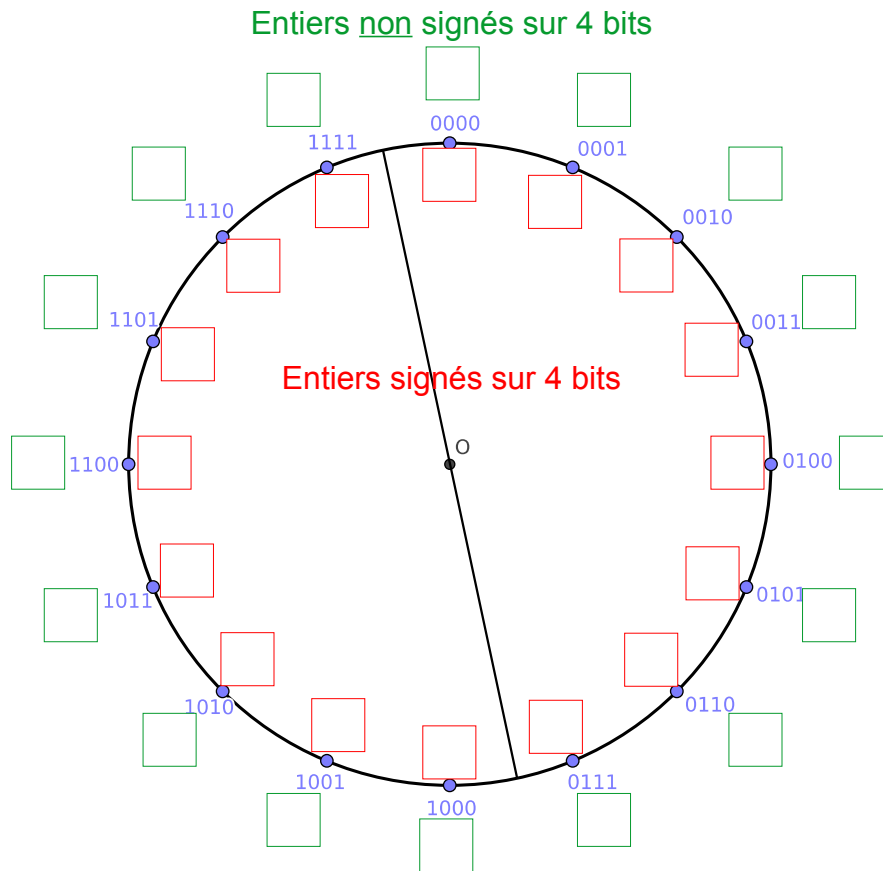
Mathématiquement : $A + (B+1) = 1\ 0000\ 0000_2$.

Mais pour un processeur 8 bits, le 9ème bit « n'existe pas » et le calcul s'écrit : $A + (B+1) = 0000\ 0000_2$.

Pour déterminer l'opposé de A, on calculera donc B+1.

Les circuits du processeur qui permettent de faire des additions vont ainsi pouvoir être utilisés aussi pour les soustractions ! (soustraction = addition avec l'opposé)

II) Roue des entiers sur 4 bits



Remarques :

- Le bit « de poids fort » d'un entier signé permet de déterminer son signe
- Sur n bits, on peut coder les entiers non signés de 0 à
et les entiers signés de à

III) Dans les exercices :

Ex 1 : Convertir en base 10 l'entier signé sur 8 bits : $1000\ 0111_2$.

Le bit de poids fort est 1 donc le nombre est négatif.

Déterminons son opposé :

Nombre de départ : $1000\ 0111$

On inverse les bits (Complément à 1) :

On ajoute 1 :

Conversion en base 10 :

Bilan : $1000\ 0111_2$ correspond à

Ex 2 : Convertir en base 10 l'entier signé sur 8 bits : $0100\ 0111_2$.

Le bit de poids fort est 0 donc le nombre est positif.

Conversion en base 10 :

Bilan : $0100\ 0111_2$ correspond à

Ex 3 : Écrire -13 en binaire signé sur 8 bits :

On converti $+13$ en binaire : $0000\ 1101$

On inverse les bits (Complément à 1) :

On ajoute 1 :

Bilan : -13 s'écrit en binaire

CODER UN RÉEL FLOTTANT EN BINAIRE

I) Écrire un nombre à virgule en binaire

1) Base 2 vers base 10

En base 10, on a par exemple :

$$123,456 = \square \times 10^2 + \square \times 10^1 + \square \times 10^0 + \square \times 10^{-1} + \square \times 10^{-2} + \square \times 10^{-3}$$

De même, en base 2, on aura :

$$10,1101_2 = \square \times 2^1 + \square \times 2^0 + \square \times 2^{-1} + \square \times 2^{-2} + \square \times 2^{-3} + \square \times 2^{-4}$$

2) Base 10 vers base 2

a) En décomposant la partie décimale en somme de puissances négatives de 2

Les premières puissances négatives de 2 sont :

2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}
0,5					

Ex : $0,15625 = \square = \square_2$

b) En effectuant des multiplications par 2

$0,15625 \times 2 = 0,3125$

$0,3125 \times 2 = 0,625$

$0,625 \times 2 = 1,25$

	=		,	
	=		,	

donc $0,15625 = \square$

Explication :

Refaisons exactement les mêmes calculs que ci-dessus mais directement en base 2 :

$0,00101 \times 10 = 0,0101$

$0,0101 \times 10 = 0,101$

$0,101 \times 10 = 1,01$

$0,01 \times 10 = 0,1$

$0,1 \times 10 = 1$

3) Nombres ayant une infinité de décimales en binaire

Ex : Convertissons 0,1 en binaire

$$0,1 \times 2 = 0,2$$

$$0,2 \times 2 = 0,4$$

$$0,4 \times 2 = 0,8$$

$$0,8 \times 2 = 1,6$$

$$0,6 \times 2 = 1,2$$

$$0,2 \times 2 = 0,4$$

La 6ème ligne est identique à la 2ème !

Le processus de conversion en base 2 ne s'arrêtera donc jamais :

$$0,1 = 0,00011001100110011\dots$$

Le nombre 0,1 a donc en binaire une infinité de chiffres après la virgule et devra donc être arrondi.

Conséquence :

Taper les 2 commandes ci-dessous dans la console Python :

```
>>> 0.1+0.2
```

```
>>> 0.1+0.2 == 0.3
```

On ne comparera donc jamais directement des flottants !

II) Norme IEEE754

1) Le problème du codage de la virgule :

On a vu précédemment comment coder 0,15625 en binaire :

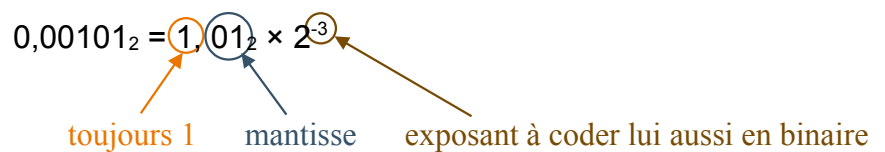
$$0,15625_{10} = 0,00101_2$$

Mais comment peut-on coder la virgule, sachant que l'ordinateur ne connaît pas d'autres symboles que le 0 et le 1 ?

Une première solution, appelée codage en **virgule fixe**, consiste à décider que tous les nombres seront codés par exemple sur 16 bits avec les 10 premiers bits pour la partie entière et les 6 derniers pour la partie décimale. Ce système permet des calculs plutôt simples mais n'est pas adapté pour manipuler à la fois des très grands nombres et des très petits. Il est donc peu utilisé de nos jours.

Du coup une autre solution est apparue, appelée codage en **virgule flottante**, qui consiste à « décaler » la virgule comme on le fait en notation scientifique :

$$0,00101_2 = (1,01_2 \times 2^{-3})$$



toujours 1 mantisse exposant à coder lui aussi en binaire

Bien sûr, le calcul en virgule flottante suppose des circuits spécialisés assez complexes, mais il permet de manipuler une plage de nombres bien plus importante que le calcul en virgule fixe, tout en gardant une meilleure précision. Du coup, la quasi totalité des processeurs travaillent aujourd'hui en virgule flottante en utilisant la norme IEEE754. Cette norme permet de coder un nombre décimal sur 32 bits (simple précision), 64 bits (double précision) ou même 128 bits (quadruple précision).

Sur la plupart des interpréteurs Python, les nombres de type float sont codés en double précision.

2) Format « simple précision » sur 32 bits

s	Exposant + 127 (8 bits)	Mantisse (23 bits)
---	-------------------------	--------------------

Bit de signe :
0 : positif
1 : négatif

Le décalage de 127 permet de
représenter aussi les exposants négatif

Ex 1 : Représenter -0,15625 en simple précision

-0,15625 est négatif donc le bit de signe est

$$0,15625 = 0,125 + 0,03125 = 2^{-3} + 2^{-5} = 0,00101_2 = 1,01_2 \times 2^{-3}$$

La mantisse est

Codons l'exposant : + 127 = = $64 + 32 + 16 + 8 + 4 = 0111\ 1100_2$

Bilan : -0,15625 s'écrit en simple précision : 1 01111100 010000000000000000000000

Ex 2 : Quel nombre s'écrit en simple précision : 0 10000001 011100000000000000000000 ?

Le bit de signe est 0 donc ce nombre est

La mantisse est

Décodons l'exposant : $10000001_2 = 2^0 + 2^7 = 1 + 128 = 129$

Or $129 - \text{} = \text{}$

Donc le nombre cherché est : $\text{}_2 \times 2^{\text{}} = (1+0,25+0,125+0,0625) \times 4 = 1,4375 \times 4 = 5,75$

Convertisseur en ligne : <https://www.h-schmidt.net/FloatConverter/IEEE754.html>

3) Format « double précision » sur 64 bits

s	Exposant + 1023 (11 bits)	Mantisse (52 bits)
---	---------------------------	--------------------

4) Format « quadruple précision » sur 128 bits

s	Exposant + 16383 (15 bits)	Mantisse (112 bits)
---	----------------------------	---------------------

CODER UN CARACTÈRE EN BINAIRE

Pour coder en binaire une chaîne de caractère, ou même un texte entier, on utilise traditionnellement une « page de code », c'est-à-dire un grand tableau qui associe à chacun des caractères que l'on compte utiliser, un numéro unique.

Ces pages de code se sont progressivement standardisées pour permettre les échanges entre ordinateurs de marques différentes, puis de pays différents.

Nous retiendrons ci-dessous trois étapes majeures de cette standardisation.

1) La table ASCII

Dans les années 1960, naît une première initiative encore aujourd'hui incontournable : La table ASCII (American Standard Code for Information Interchange).

Cette table utilise 7 bits et permet donc de coder 128 caractères différents :

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Remarques :

- Les codes ASCII vont de 0x00 à 0x7F.
Par exemple le code 0x21 correspond au point d'exclamation.
- Pour passer des majuscules aux minuscules et vice-versa, il suffit d'ajouter ou de retrancher 0x20.
- Pour coder un chiffre, il suffit d'ajouter 0x30 à la valeur du chiffre.
- Le code 0x20 correspond à l'espace blanc.
- Les deux premières lignes correspondent à des caractères non imprimables « de contrôle » dont la plupart ne sont plus utilisés de nos jours.

Exercice :

La fonction `chr(i)` de Python permet d'obtenir le caractère de code `i`.

En tapant les commandes ci-dessous dans la console Python, essayer de deviner le rôle des caractères de contrôle correspondants :

Commande	Caractère de contrôle	Signification
<code>print("toto"+chr(8)+"a")</code>	BS	
<code>print("toto"+chr(9)+"a")</code>	HT	
<code>print("toto"+chr(10)+"a")</code>	LF	
<code>print("toto"+chr(13)+"a")</code>	CR	

II) La norme ISO 8859-1

La table ASCII convient bien aux anglophones mais ne contient aucun caractère accentué.

Aussi, quand les processeurs 8 bits se sont généralisés dans les années 70-80, on a cherché à tirer profit du 8ème bit non utilisé par ASCII pour ajouter des caractères supplémentaires.

Ainsi est née en 1986 la norme « ISO 8859-1 » souvent appelée « latin-1 » et qui a été très utilisée par les pays occidentaux.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
8	PAD	HOP	BPH	NBH	IND	NEL	SSA	ESA	HTS	HTJ	VTJ	PLD	PLU	RI	SS2	SS3
9	DCS	PU1	PU2	STS	CCH	MW	SPA	EPA	SOS	SGCI	SCI	CSI	ST	OSC	PM	APC
A	Nbsp	ı	ø	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
B	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Remarques :

- Cette nouvelle table reprend intégralement la table ASCII.
- Le fait d'ajouter un seul bit permet de doubler le nombre de caractères disponibles.
- Il y a toujours un écart de 0x20 entre les majuscules et les minuscules accentuées.
- Il manque le caractère € qui n'existait pas encore à l'époque.

Exercice :

Une adresse mémoire de mon ordinateur contient 1110 1010.

À quoi peut correspondre cette valeur dans les cas suivants :

- C'est un entier non signé ?
- C'est un entier signé sur 8 bits ?
- C'est un caractère encodé en latin-1 ?

III) La norme Unicode

La table ISO 8859-1 convenait bien pour les américains et les européens de l'ouest mais ne permettait pas à une personne utilisant par exemple une version de Windows française de lire directement un texte grec ou russe.

Avec l'avènement d'Internet et la possibilité d'atteindre en un clic des sites web écrits dans d'autres alphabets, une nouvelle norme appelée « Unicode » a donc été créée dès les années 90 avec pour objectif de pouvoir encoder tous les alphabets et symboles connus.

[Le début de la table Unicode sur Wikipedia](#)

Remarques :

- Dans la table Unicode, chaque caractère a un numéro appelé « point de code » compris entre 0x0 et 0x10FFFF, soit plus d'un million de possibilités !
- Actuellement, environ 150000 caractères sont définis. Il y a donc encore beaucoup de place disponible pour de nouveaux caractères ou symboles.
- La table Unicode commence en reprenant intégralement la table latin-1 (qui elle même commence en reprenant la table ASCII).
- Python 3 utilise par défaut Unicode
- Les fonctions `chr()` et `ord()` de Python permettent d'obtenir le point de code d'un caractère Unicode et réciproquement.

Encodage UTF-8 :

En latin-1 et ASCII, un caractère prend un octet en mémoire. En revanche, si l'on choisit d'encoder un caractère Unicode par son point de code, il prendra 3 octets en mémoire !

On préfère donc utiliser « l'UTF-8 » qui est une astuce permettant de représenter **tous** les caractères Unicode en utilisant de 1 à 4 octets selon les caractères (un octet pour le début de la table, c'est à dire pour les caractères les plus utilisés et jusqu'à 4 octets pour la fin de la table, c'est à dire les caractères les moins utilisés).

Pour encoder un caractère en UTF8, on regarde donc combien de bits sont nécessaires pour convertir son point de code en binaire :

Nombre de bits	Codage en UTF8
0 → 7	0XXX XXXX
8 → 11	110X XXXX - 10XX XXXX
12 → 16	1110 XXXX - 10XX XXXX - 10XX XXXX
17 → 21	1111 0XXX - 10XX XXXX - 10XX XXXX - 10XX XXXX

Exemples :

Caractère	Point de code : hex → bin	#bits	Codage UTF8
A	41 → 0100 0001	7	0100 0001
à	E0 → 1110 0000	8	1100 0011 1010 0000
%	2030 → 0010 0000 0011 0000	14	1110 0010 1000 0000 1011 0000

ALGÈBRE BOOLÉENNE

I) Un peu d'histoire

1) George Boole

Vers les années 1850, le mathématicien britannique George Boole entreprend de refonder mathématiquement la logique, notamment sous la forme de calculs avec des 0 et des 1.

Ainsi « vrai » et « faux » sont notés respectivement : 1 et 0.

« a ou b » est noté : « $a + b$ »

« a et b » est noté : « $a . b$ »

La négation de a est notée : $\bar{a}$ ou encore « non a »

Comme il y a en général peu de cas possibles, on peut expliciter les opérations booléennes dans des « tables de vérité » :

a	b	$a + b$
0	0	
0	1	
1	0	
1	1	

a	b	$a . b$
0	0	
0	1	
1	0	
1	1	

a	$\bar{a}$
0	
1	

Exemples d'application :

Ex 1 : « Si j'ai faim ou soif, alors j'ouvrirai le frigo ».

D'après le tableau de gauche, si on a faim et pas soif, on ouvrira quand même le frigo.

Ex 2 : « Si j'ai assez d'argent et que la boulangerie est ouverte, alors j'irai acheter un croissant ».

D'après le tableau du milieu, on ne pourra acheter un croissant que si les deux conditions sont vérifiées.

2) Claude Shannon et George Stibitz

Un peu moins de 100 ans plus tard, vers 1937, il se trouve que les ingénieurs Claude Shannon et George Stibitz ont eu à peu près en même temps l'idée d'utiliser des relais (sorte d'interrupteurs commandés électriquement) pour réaliser des montages électriques permettant de faire des opérations booléennes. Ce sera le début d'une profusion de recherches et d'expérimentations qui vont aboutir très rapidement aux premiers ordinateurs.

« ET » en utilisant deux interrupteurs en série



Le courant ne passe que si les deux interrupteurs sont fermés

« OU » avec deux interrupteurs en parallèle



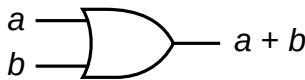
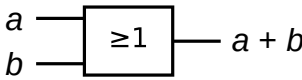
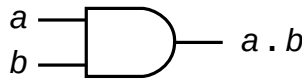
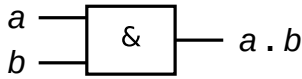
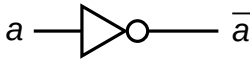
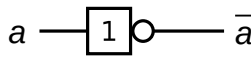
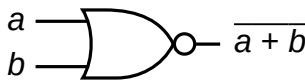
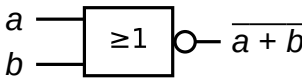
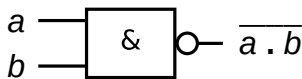
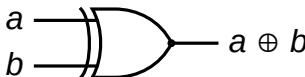
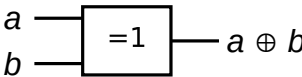
Le courant passe dès que l'un des interrupteurs est fermé

Les relais montrant vite leurs limites seront remplacés par des tubes à vides, puis par des transistors (1947). Puis ces transistors seront gravés dans des circuits intégrés (1959).

La technologie a donc beaucoup évolué depuis cette époque, mais sur le plan théorique, aujourd'hui encore, un processeur est constitué essentiellement d'interrupteurs commandés électriquement interconnectés les uns aux autres !

II) Les principales portes logiques

Une porte logique est un circuit électronique qui permet d'implémenter une opération booléenne.
Le 0 désigne une tension nulle et le 1 désigne une tension en général égale à 5V.

Porte	Symbole américain	Symbole européen	Table de vérité															
OU			<table><tr><th>a</th><th>b</th><th>a + b</th></tr><tr><td>0</td><td>0</td><td></td></tr><tr><td>0</td><td>1</td><td></td></tr><tr><td>1</td><td>0</td><td></td></tr><tr><td>1</td><td>1</td><td></td></tr></table>	a	b	a + b	0	0		0	1		1	0		1	1	
a	b	a + b																
0	0																	
0	1																	
1	0																	
1	1																	
ET			<table><tr><th>a</th><th>b</th><th>a . b</th></tr><tr><td>0</td><td>0</td><td></td></tr><tr><td>0</td><td>1</td><td></td></tr><tr><td>1</td><td>0</td><td></td></tr><tr><td>1</td><td>1</td><td></td></tr></table>	a	b	a . b	0	0		0	1		1	0		1	1	
a	b	a . b																
0	0																	
0	1																	
1	0																	
1	1																	
NON (inverseur)			<table><tr><th>a</th><th>ā</th></tr><tr><td>0</td><td></td></tr><tr><td>1</td><td></td></tr></table>	a	ā	0		1										
a	ā																	
0																		
1																		
NON OU (NOR)			<table><tr><th>a</th><th>b</th><th>ā + b</th></tr><tr><td>0</td><td>0</td><td></td></tr><tr><td>0</td><td>1</td><td></td></tr><tr><td>1</td><td>0</td><td></td></tr><tr><td>1</td><td>1</td><td></td></tr></table>	a	b	ā + b	0	0		0	1		1	0		1	1	
a	b	ā + b																
0	0																	
0	1																	
1	0																	
1	1																	
NON ET (NAND)			<table><tr><th>a</th><th>b</th><th>ā . b</th></tr><tr><td>0</td><td>0</td><td></td></tr><tr><td>0</td><td>1</td><td></td></tr><tr><td>1</td><td>0</td><td></td></tr><tr><td>1</td><td>1</td><td></td></tr></table>	a	b	ā . b	0	0		0	1		1	0		1	1	
a	b	ā . b																
0	0																	
0	1																	
1	0																	
1	1																	
OU Exclusif (XOR)			<table><tr><th>a</th><th>b</th><th>a ⊕ b</th></tr><tr><td>0</td><td>0</td><td></td></tr><tr><td>0</td><td>1</td><td></td></tr><tr><td>1</td><td>0</td><td></td></tr><tr><td>1</td><td>1</td><td></td></tr></table>	a	b	a ⊕ b	0	0		0	1		1	0		1	1	
a	b	a ⊕ b																
0	0																	
0	1																	
1	0																	
1	1																	

III) Propriétés

1) Commutativité : l'ordre des termes n'a pas d'importance

$$a + b = b + a$$

$$a \cdot b = b \cdot a$$

2) Associativité : certaines parenthèses sont inutiles

$$a + (b + c) = (a + b) + c = a + b + c$$

$$a \cdot (b \cdot c) = (a \cdot b) \cdot c = a \cdot b \cdot c$$

3) Distributivité

Distributivité de la multiplication par rapport à l'addition : $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$

Distributivité de l'addition par rapport à la multiplication : $a + (b \cdot c) = (a + b) \cdot (a + c)$

4) Divers

$$a + a = \boxed{}$$

$$a \cdot a = \boxed{}$$

$$\overline{\overline{a}} = \boxed{}$$

$$a + \overline{a} = \boxed{}$$

$$a \cdot \overline{a} = \boxed{}$$

5) Formules de Morgan

$$\overline{a + b} = \overline{a} \cdot \overline{b}$$

Ex : La négation de « je vais prendre un thé ou un café » est :

$$\overline{a \cdot b} = \overline{a} + \overline{b}$$

Ex : La négation de « je vais manger et boire » est :

IV) Démonstrations à l'aide de tables de vérité

Faire une table de vérité permet de lister tous les cas possibles et donc de démontrer certaines propriétés.

Ex 1 : Démontrer que : $a \oplus b = a \cdot \bar{b} + \bar{a} \cdot b$

a	b	$a \oplus b$	$\bar{a}$	$\bar{b}$	$a \cdot \bar{b}$	$\bar{a} \cdot b$	$a \cdot \bar{b} + \bar{a} \cdot b$
0	0	☐	☐	☐	☐	☐	☐
0	1	☐	☐	☐	☐	☐	☐
1	0	☐	☐	☐	☐	☐	☐
1	1	☐	☐	☐	☐	☐	☐

Ex 2 : Démontrer que : $a + (b \cdot c) = (a + b) \cdot (a + c)$

a	b	c	$b \cdot c$	$a + (b \cdot c)$	$a + b$	$a + c$	$(a + b) \cdot (a + c)$
0	0	0	☐	☐	☐	☐	☐
0	0	1	☐	☐	☐	☐	☐
0	1	0	☐	☐	☐	☐	☐
0	1	1	☐	☐	☐	☐	☐
1	0	0	☐	☐	☐	☐	☐
1	0	1	☐	☐	☐	☐	☐
1	1	0	☐	☐	☐	☐	☐
1	1	1	☐	☐	☐	☐	☐

V) Pour les curieux (ne pas recopier sur votre cours)

1) Le premier montage de George Stibiz :

Additionneur 1 bit appelé Model-K



2) Réaliser des portes logiques avec des transistors cmos :

<http://ressources.univ-lemans.fr/AccesLibre/UM/Pedago/physique/02/electro/logiCmos.html>

Simulations : [Transistors Mos de type N et P](#) – [Inverseur](#) – [Non ET](#) – [Non OU](#)

3) Additionneur 4 bits réalisé avec des transistors :

<https://www.youtube.com/watch?v=WRGsdPUG2Gc>

ARCHITECTURE D'UN ORDINATEUR

1) Architecture de « Von Neumann »

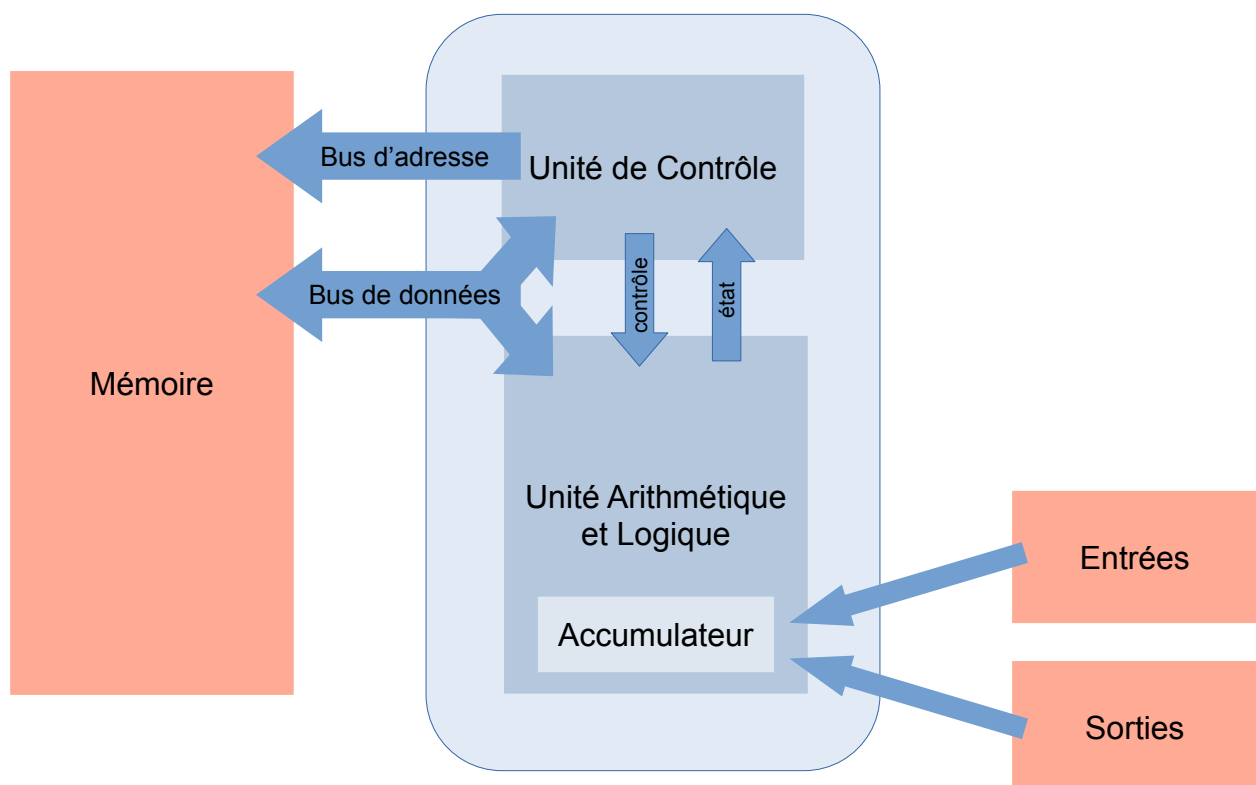
Dans les années 1940, apparaissent les premiers calculateurs électroniques. Ces calculateurs n'ont ni clavier, ni souris, ni écran et il n'y a pas encore de véritables langages de programmation.

Pour exécuter un nouveau programme, il faut manipuler de très nombreux interrupteurs ou modifier le branchement d'un grand nombre de câbles et cela prend plusieurs heures à toute une équipe.

En 1945, des ingénieurs travaillant sur un nouveau projet d'ordinateur appelé EDVAC publient une note proposant deux innovations majeures qui feront date dans l'histoire de l'informatique et qui seront reprises dans la plupart des projets qui suivront :

- Le programme est désormais enregistré dans la mémoire et il n'y aura donc plus besoin de modifier le câblage de la machine. La mémoire contient donc à la fois le programme et les données.
- À l'**Unité Arithmétique et Logique** (UAL) qui fait les calculs, on ajoute une **Unité de Contrôle** (UC), qui lit les instructions du programme en mémoire et pilote l'UAL.

C'est cette nouvelle architecture que l'on appelle aujourd'hui encore « architecture de Von Neumann ».



En savoir plus sur l'aspect historique :

<https://interstices.info/le-modele-darchitecture-de-von-neumann/>

II) Langage machine et assembleur.

Un processeur exécute des instructions qui sont des suites de 0 et de 1.

Exemple, avec le [simulateur AQA](#) pour faire l'addition : Registre1+Registre2 vers Registre3

Instruction langage machine	1110 0000 1000	0001	0011	0000 0000	0010
	Addition de registre à registre	N° registre opérande 1	N° registre résultat	Inutilisé	N° registre opérande 2
Assembleur	add R3, R1, R2				

Les instructions acceptées par un processeur sont en nombre en général assez réduit et elles sont directement câblées dans les différents circuits électroniques du processeur.

On y trouve :

- Des instructions de transfert de données entre la mémoire et les registres
- Des opérations arithmétiques (les 4 opérations) ou logique (opérateurs booléens)
- Des instructions de contrôle (branchement, branchement conditionnel,...)

III) L'Unité de Contrôle

L'unité de contrôle est le chef d'orchestre du processeur. Elle exécute en permanence une boucle sans fin :

- 1) **Fetch** : L'unité de contrôle récupère l'adresse mémoire de la prochaine instruction à exécuter. Cette adresse est stockée dans le registre PC (Programme Counter). Puis elle lit en mémoire la valeur de cette instruction et la stocke dans le registre CIR (Current Instruction Register). Ensuite, elle incrémente le registre PC.
- 2) **Decode** : L'unité de contrôle identifie de quel type d'instruction il s'agit et récupère les données sur lesquelles cette instruction doit opérer (registre, valeurs en mémoire...)
- 3) **Execute** : L'unité de contrôle exécute l'instruction. Par exemple, s'il s'agit d'une multiplication, elle la fait calculer par l'UAL, s'il s'agit d'un branchement, elle met à jour la valeur de PC...

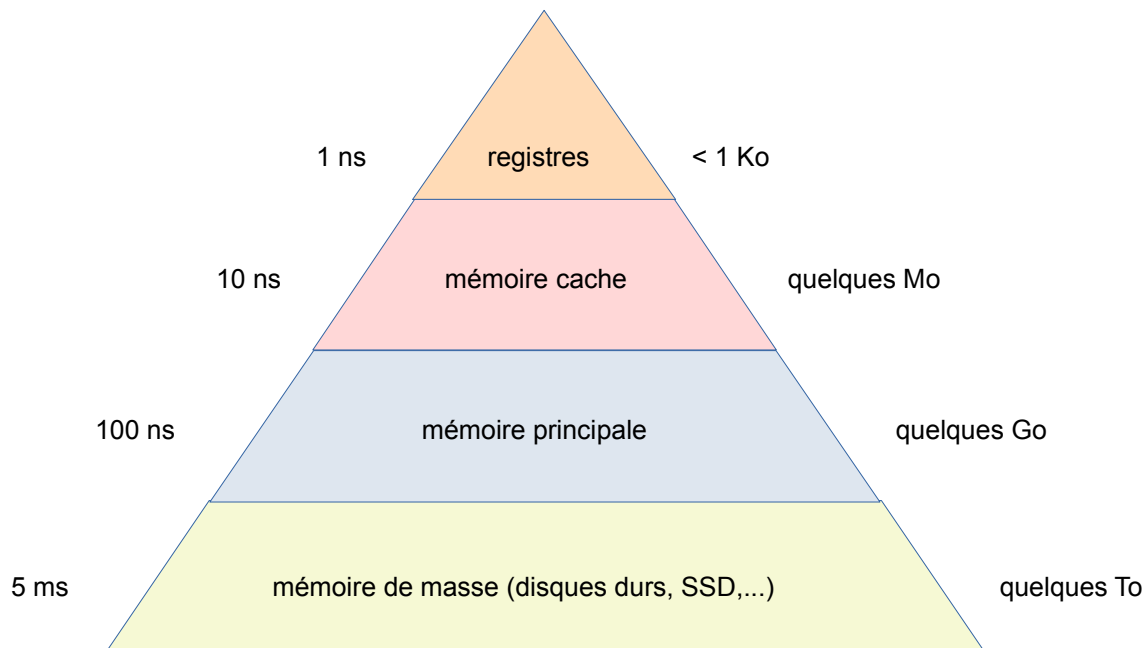
IV) L'Unité Arithmétique et Logique

L'Unité Arithmétique et Logique s'occupe des calculs et contient des circuits spécifiques pour :

- Exécuter les quatre opérations sur les entiers
- Exécuter les quatre opérations sur les nombres en virgule flottante
- Faire des comparaisons
- Exécuter les différentes fonctions booléennes (et, ou, non, ou exclusif...)
- Faire des opérations sur les bits (décalage à droite ou à gauche...)

V) La mémoire

La mémoire principale est beaucoup plus lente que le processeur. Pour éviter que ce dernier ne soit trop ralenti par les accès mémoire, on intercale une mémoire rapide (mais chère!) appelée « mémoire cache » où un contrôleur de mémoire copie les instructions et données qui ont de fortes chances d'être utilisées prochainement par le processeur.



On distingue également :

La mémoire **vive** ou **volatile** appelée également **RAM** (Random Access Memory) dont le contenu est perdu dès que l'ordinateur est éteint.

La mémoire **morte** ou **non volatile** appelée également **ROM** (Read Only Memory) dont le contenu ne s'efface pas lorsque l'ordinateur est éteint. Par exemple, le programme qui se lance au démarrage de l'ordinateur (BIOS ou UEFI) est stocké en mémoire morte et il a notamment pour rôle d'aller chercher sur le disque dur les fichiers de démarrage du système d'exploitation de l'ordinateur pour les copier dans la mémoire vive.

VI) L'horloge

Le processeur est cadencé par une horloge. A chaque impulsion de cette horloge, il exécute une instruction ou une partie d'instruction. La durée entre deux impulsions est choisie pour laisser le temps aux signaux électriques de se propager dans tout le processeur.

Si l'on augmente la fréquence de l'horloge (overclocking), le processeur calcule plus vite mais risque de faire des erreurs car les signaux n'auront pas toujours le temps d'arriver au bon endroit. De plus, les transistors utilisés dans les circuits intégrés consomment surtout au moment où ils « commutent » c'est à dire passent du niveau électrique 0 à 1 ou 1 à 0. Plus un processeur va vite, plus ses transistors commutent souvent, plus le processeur chauffe et peut même être détruit !

VII) Quelques ordres de grandeur

Exemple avec un processeur Intel® Core™ i5 :

- Jeu d'instructions : x86-64 (environ 200 instructions)
- Nombre de transistors : 1,3 milliards
- Registres : Une vingtaine de registres 64 bits
- Fréquence d'horloge : 4 Ghz
- Mémoire cache : 24 Mo
- Nombre de cœurs : 10
- Précision de la gravure : 10 nm
- Prix : 200 €

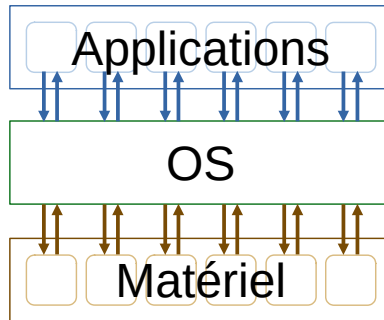
En savoir plus :

- Sur les circuits intégrés en général : <https://www.youtube.com/watch?v=ee-LhNZPZ1U>
- Sur la fabrication des processeurs : <https://www.youtube.com/watch?v=NFr-WyytNfo>

SYSTÈMES D'EXPLOITATIONS

I) Rôle d'un système d'exploitation

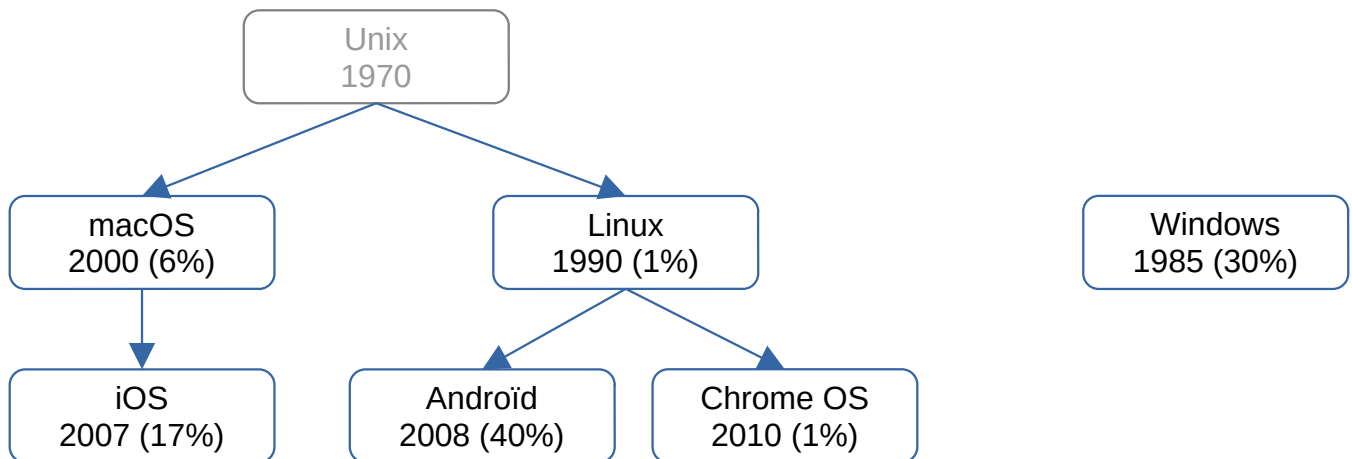
Dans les années 1950, un ordinateur ne pouvait exécuter qu'un seul programme à la fois. De plus, un programme devait être en partie réécrit pour pouvoir être exécuté sur un autre ordinateur. Assez vite, on a donc eu l'idée d'ajouter un logiciel intermédiaire qui serve d'interface entre les différents programmes à exécuter et l'ordinateur lui-même. C'est ainsi que sont nés les premiers systèmes d'exploitation (Operating System – OS).



De nos jours, un OS permet notamment :

- D'exécuter en parallèle plusieurs applications en même temps (multitâche)
- De gérer plusieurs utilisateurs (multi-utilisateur).
- De masquer les particularités matérielles de la machine sur laquelle il est installé en faisant appel à des concepts plus « abstraits » (par exemple la notion de fichier est une notion abstraite indépendante de la façon dont les données sont concrètement stockées sur le support physique).

Les principaux OS aujourd'hui :



(Statistiques d'après <https://gs.statcounter.com/os-market-share>)

L'OS est chargé au démarrage de l'ordinateur et tourne ensuite en permanence. Il est habituellement composé :

- D'un noyau (kernel)
- De bibliothèques (API) permettant aux applications d'accéder au matériel et aux services offerts par l'OS
- D'une interface graphique (GUI) et/ou d'une interface en ligne de commande (CLI)
- D'un ensemble d'outils systèmes

(Pour en savoir plus sur l'histoire d'Unix : <https://www.youtube.com/watch?v=Za6vGTLp-wg>)

II) Le noyau

Le noyau gère notamment :

- Les processus (process)
Un processus est un programme en cours d'exécution. Le noyau alloue à chaque processus une partie de la mémoire et une partie du temps d'utilisation du processeur. Il redirige les entrées-sorties vers les processus auxquels elles sont destinées...
- Le matériel (hardware)
La mémoire (cache, mémoire principale), le processeur (cœurs), les périphérique de stockage (disque dur, SSD, DVD), les périphérique d'entrées-sorties (clavier, souris, écran, imprimante), les périphériques de communication (réseau Ethernet et Wifi, Bluetooth, USB).
Chaque périphérique est piloté par un logiciel appelé « pilote » (driver).
- Les systèmes de fichiers (file system)
Un système de fichiers (FAT32, NTFS, Ext4, APFS) est une façon d'organiser et de formater les fichiers et dossiers sur un support de stockage (disque dur, clé USB,...)
- Les différents protocoles réseau (TCP-IP,...)
- Les sessions utilisateurs avec les droits associés

III) Les bibliothèques

Les applications n'accèdent pas au noyau directement, mais via un ensemble de fonctions (appels système) regroupés dans des bibliothèques (API).

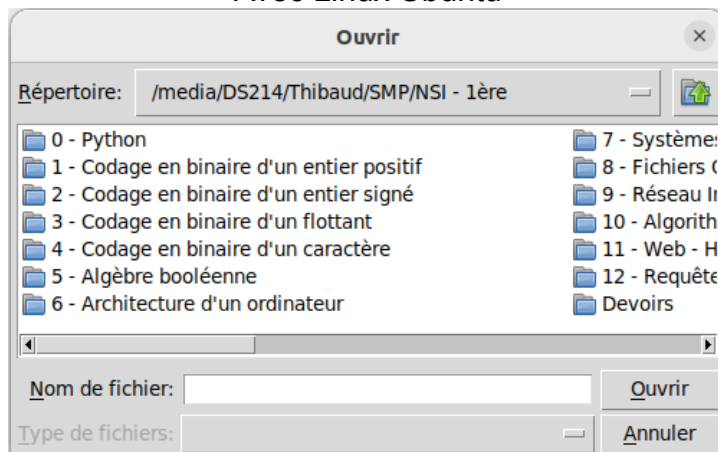
Exemple 1 : Si on exécute le code Python ci-dessous, l'interpréteur python va faire trois appels système : un pour ouvrir toto.txt, un pour lire son contenu et un pour le fermer. Le même code source python peut donc fonctionner sans modification sur des OS différents et avec des systèmes de fichiers différents. Le programmeur n'a pas besoin de connaître le fonctionnement interne du système de fichiers utilisé, ni même de savoir lequel est utilisé !

```
fichier = open("toto.txt")
texte = fichier.readlines()
fichier.close()
```

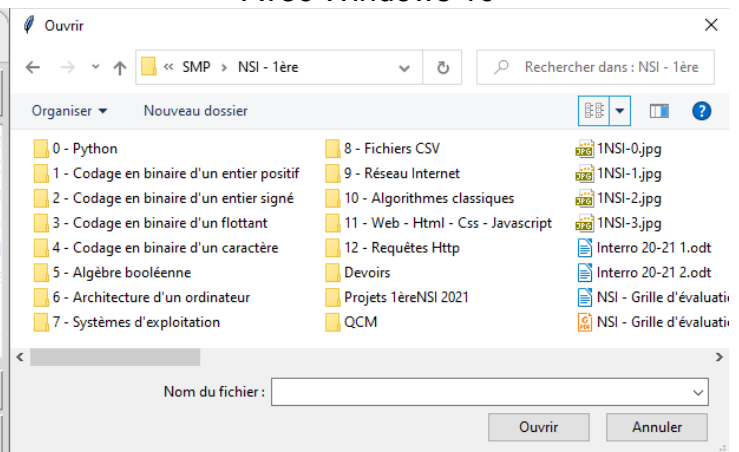
Exemple 2 : De même, ce ne sont pas les applications qui créent elles-mêmes les fenêtres, boutons et menus dont elles ont besoin pour leurs interfaces utilisateur. Elles vont plutôt faire des appels systèmes pour demander à l'OS de créer ces éléments pour elles.

Ainsi avec Python, la même fonction `tkinter.filedialog.askopenfilename()` n'a pas tout à fait la même allure selon l'OS utilisé.

Avec Linux Ubuntu

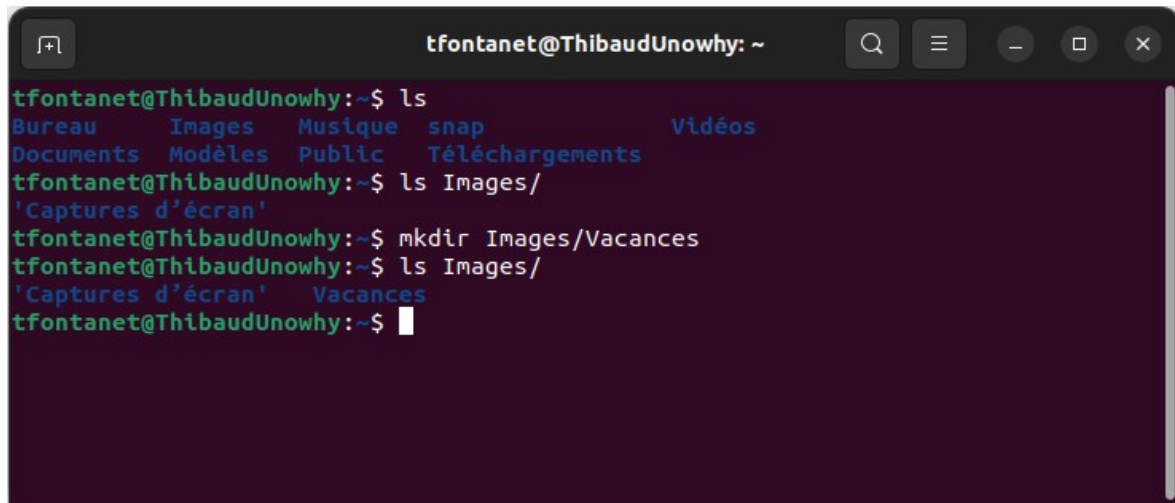


Avec Windows 10



IV) Interface en « Ligne de commande »

Les premières interfaces graphiques avec fenêtres, boutons, icônes et utilisation de la souris... se démocratisent dans les années 1980. Auparavant, l'utilisateur ne pouvait interagir avec les applications et l'OS que via ce que l'on appelle « la ligne de commande ».



```
tfontanet@ThibaudUnowhy: ~  
tfontanet@ThibaudUnowhy:~$ ls  
Bureau  Images  Musique  snap      Vidéos  
Documents  Modèles  Public  Téléchargements  
tfontanet@ThibaudUnowhy:~$ ls Images/  
'Captures d'écran'  
tfontanet@ThibaudUnowhy:~$ mkdir Images/Vacances  
tfontanet@ThibaudUnowhy:~$ ls Images/  
'Captures d'écran'  Vacances  
tfontanet@ThibaudUnowhy:~$
```

Curieusement, 40 ans plus tard, la ligne de commande existe toujours ! En effet, de nombreuses tâches répétitives ou d'administration se font beaucoup plus efficacement avec la ligne de commande qu'avec une interface graphique. De nombreux serveurs ou matériels embarqués ne s'administrent qu'en ligne de commande (et souvent à distance).

A noter que l'on peut rassembler plusieurs commandes dans un script et que ces scripts peuvent devenir de vrais programmes avec variables, boucles, tests, fonctions,... !

LE RÉSEAU INTERNET

I) Généralités sur Internet

1) Internet, le réseau qui interconnecte les autres

Très vite dans l'histoire de l'informatique, on a cherché à relier entre eux des ordinateurs. Dès les années 60 différentes solutions techniques apparaissent permettant de créer des réseaux entre plusieurs ordinateurs mais ces solutions ne sont pas compatibles entre elles. Internet est né de la volonté d'interconnecter le plus de réseau possible et s'est progressivement imposé vers les années 1990 à 2000.

Quelques caractéristiques d'Internet aujourd'hui :

- Internet n'est pas un grand réseau homogène mais plutôt l'interconnexion d'un très grand nombre de réseaux locaux.
- Internet permet de relier entre eux des terminaux de natures très différentes : ordinateurs, smartphones, imprimantes, montres connectées,...
- Internet utilise des moyens très différents pour faire transiter les données : câbles Ethernet, fibres, ondes Wi-Fi, 4G ou 5G, câbles sous-marin, liaisons satellite.
Voici par exemple une carte des câbles sous-marins : <https://www.submarinecablemap.com/>
- Internet a une structure très décentralisée : Si certains équipements tombent en panne, d'autres prennent le relais automatiquement.

2) Réseaux LAN et WAN

Les **réseaux locaux** (souvent appelés **LAN** = Local Area Network) forment les briques de base du réseau internet. Un LAN peut contenir quelques terminaux (par exemple le réseau familial est un LAN) ou bien quelques milliers (réseaux d'universités ou d'entreprises par exemple).

Dans un LAN, les ordinateurs sont tous reliés entre eux via :

- Un ou plusieurs **switchs** (appelés en français **commutateurs**) qui permettent de connecter des ordinateurs avec des câbles **Ethernet** et des prises **RJ45**.
- Un ou plusieurs **points d'accès Wifi**.



Un LAN étant local :

- Il a souvent des liaisons très rapides.
- Il est en général privé avec un enjeu de sécurité.
- Il y a une personne qui en maîtrise l'architecture globale (en général simple) et qui peut intervenir rapidement en cas de panne.

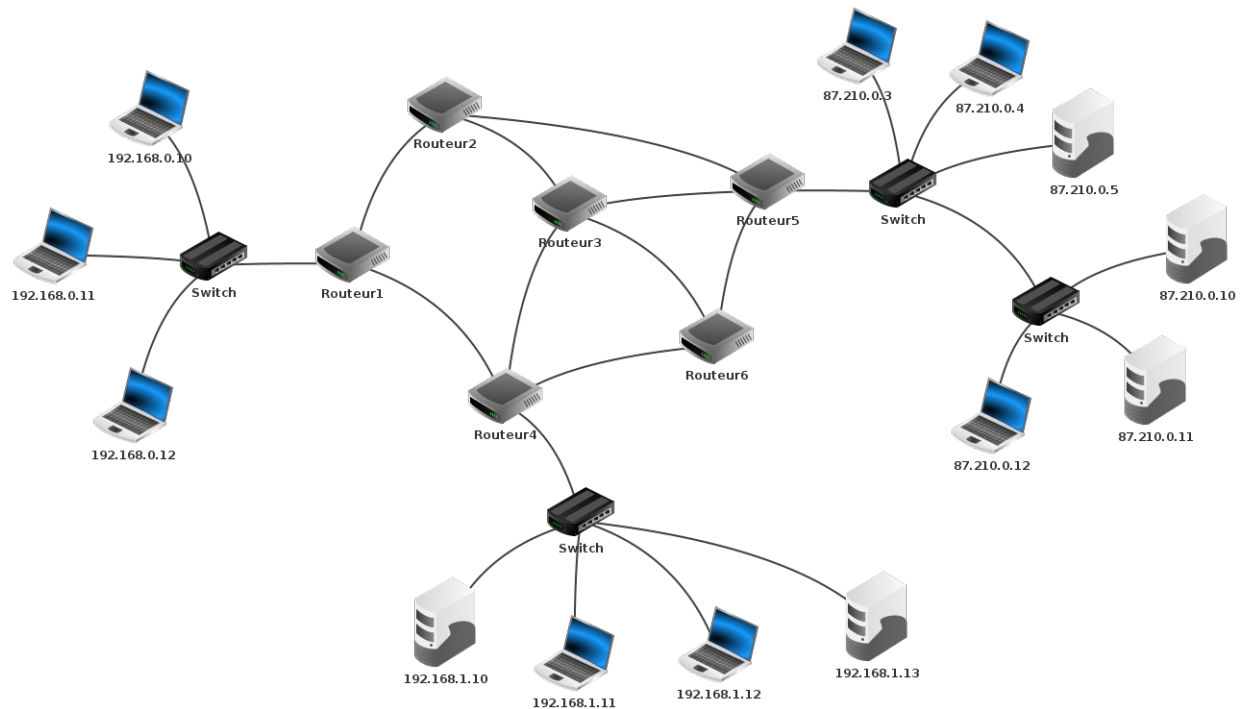
Communication entre deux ordinateurs :

- Si un ordinateur veut communiquer avec un ordinateur qui est dans le même LAN, cette communication peut se faire directement sans passer par d'autres intermédiaires que les switchs et les points d'accès wifi du LAN.
- En revanche, si cet ordinateur doit communiquer avec un ordinateur qui est sur un autre LAN, cette communication doit passer par la porte de sortie du LAN appelée **passerelle** ou **routeur**, puis sauter de routeur en routeur jusqu'à atteindre le routeur qui lui permettra d'entrer dans le LAN de l'autre ordinateur. Cette communication traverse alors ce que l'on appelle le **réseau étendu** (souvent appelé **WAN** = Wide Area Network).
- Les box familiales ont comme particularité de regrouper en un seul boîtier : un switch (souvent 4 ports LAN en RJ45), un point d'accès wifi et un routeur (1 port WAN en RJ45 ou fibre optique).

Livebox Play



Combien de LAN différents comptez-vous ci-dessous ? 3 ? 4 ? 12 ?
Y a-t-il un WAN ?



3) Adresses IP et adresses MAC

Sur internet, chaque **carte réseau** (appelée aussi **interface**) a deux adresses.

- Une adresse physique de 6 octets, en principe non modifiable, appelée **adresse MAC**.
Ex : A4:26:49:F9:52:D1
- Une adresse logique de 4 octets, modifiable, appelée **adresse IP**.
Ex : 192.168.0.163

Remarques :

- L'adresse MAC est définie par le constructeur de la carte réseau lors de sa fabrication de telle sorte qu'il n'y ait pas 2 cartes réseau qui aient la même adresse MAC.
- L'adresse IP est modifiable par l'administrateur de l'ordinateur. Sa structure permet aux routeurs de savoir dans quelle direction envoyer les données (un peu comme une adresse postale).
- Un ordinateur peut avoir plusieurs cartes réseau. Par exemple de nombreux ordinateurs portables ont à la fois une carte Ethernet avec prise RJ45 et une carte Wifi. Ces ordinateurs ont alors autant d'adresses IP et d'adresses MAC que de cartes réseau.
- Si votre smartphone est connecté au Wifi, alors vous trouverez son adresse IP et son adresse MAC dans les paramètres. En revanche, s'il est connecté sur le réseau GSM, vous verrez qu'il a toujours une IP (différente de celle du Wifi) mais pas d'adresse MAC car ce type de réseau utilise un autre identifiant unique (IMSI) lié à votre carte SIM.
- Les adresses IP à 4 octets (ipv4) permettent d'adresser un peu plus de 4 milliards de terminaux. Ce n'est plus assez ! Un certain nombre de techniques ont été mises en place pour retarder la pénurie (adresses privées avec NAT et PAT) mais surtout une nouvelle norme d'adresses IP à 16 octets (ipv6) se met en place progressivement.

Pourquoi deux adresses ?

- Les adresses MAC sont un peu comme des empreintes digitales. Elles sont uniques et permettent d'être certain que l'on s'adresse à la bonne personne.
- Les adresses IP sont un peu comme des adresses postales où l'octet le plus à gauche serait le pays, puis la ville, puis la rue et enfin l'octet le plus à droite le numéro de la maison. Elles permettent aux routeurs de savoir dans quelle direction acheminer les messages qu'ils reçoivent. Quand on « déménage » (changement d'abonnement internet, changement de service dans une entreprise,...), on change d'adresse IP.
Pour voir par exemple les plages d'adresses IP attribuées à votre fournisseur internet : <https://bgp.he.net/>

4) Masques de sous-réseau

Quand on configure l'adresse IP d'une carte réseau, on précise en même temps un **masque de sous-réseau** et l'adresse IP de la passerelle qui permet de sortir du LAN.

Qu'est-ce qu'un masque de sous-réseau ?

- Un masque de sous-réseau est commun à tous les ordinateurs d'un même LAN. Il permet à chaque ordinateur de savoir si une adresse IP quelconque appartient à un ordinateur du même LAN que lui ou non. En effet, si un ordinateur veut envoyer un message à une IP du même LAN, il peut l'envoyer directement (les switchs du LAN sauront envoyer le message directement au bon ordinateur). Sinon, l'ordinateur doit faire transiter le message via la passerelle qui permet de sortir du LAN.
- Le masque de sous-réseau commence toujours en binaire par une série de 1 et il termine toujours par une série de 0. Les 1 permettent de délimiter la partie de l'adresse IP commune à tous les ordinateurs du sous-réseau.

Ex 1 : Supposons qu'un ordinateur A ait la configuration ci-dessous :

	En décimal :	En binaire :
Adresse IP :	192.168. 0 .7	1100 0000 . 1010 1000 . 0000 0000 . 0000 0111
Masque de sous-réseau	255.255.255.0	1111 1111 . 1111 1111 . 1111 1111 . 0000 0000
Passerelle	192.168. 0 .1	1100 0000 . 1010 1000 . 0000 0000 . 0000 0001

Dans l'exemple ci-dessus, les bits des 3 premiers octets du masque de sous-réseau sont à 1. Cela veut dire que tous les ordinateurs dont l'IP est de la forme 192.168.0.xxx appartiennent au même LAN que l'ordinateur A.

Remarque : La passerelle doit appartenir au même LAN que l'ordinateur A, sinon ce dernier ne pourrait rien lui envoyer.

Ex 2 : Supposons qu'un ordinateur B ait la configuration ci-dessous :

	En décimal :	En binaire :
Adresse IP :	192.100. 1 .7	1100 0000 . 0110 0100 . 0000 0001 . 0000 0111
Masque de sous-réseau	255.255.252.0	1111 1111 . 1111 1111 . . 0000 0000
Passerelle	192.100. 0 .1	1100 0000 . 0110 0100 . 0000 0000 . 0000 0001

Compléter le tableau ci-dessus, puis préciser la plage d'adresses IP appartenant au même LAN.

La passerelle proposée est-elle visible par l'ordinateur B ?

Remarques :

- Pour l'ordinateur A, on voit que les 24 premiers bits du masque de sous-réseau sont à 1. Plutôt que d'écrire : 192.168.0.7 / 255.255.255.0, on préférera souvent écrire 192.168.0.7 / 24.
- Chez vous il est probable que personne ne se soit posé toutes ces questions car votre box familiale contient également ce que l'on appelle un **serveur DHCP** qui attribue automatiquement une adresse IP, un masque de sous-réseau et la passerelle à tous les ordinateurs que l'on branche sur le réseau familial. Un tel serveur existe aussi à l'école.

A faire :

Soit l'adresse IP : 172.16.5.10 / 28

- Quel est le masque de sous-réseau correspondant ?
- Combien peut-on mettre de machines différentes dans ce réseau ?

Soit l'adresse IP : 45.178.30.1 / 255.255.0.0

- Quelle est la plage d'adresses IP visibles par cette adresse ?
- Réécrire cette adresse IP plus simplement.

5) Ports

Sur un ordinateur, il y a souvent plusieurs applications différentes qui utilisent le réseau simultanément. Si on est par exemple en train de charger 2 pages web différentes tout en regardant une vidéo youtube et que le service de mise à jour de l'ordinateur se met en route, cela fait 4 « utilisations » simultanées d'internet. Pour que l'ordinateur sache à laquelle transmettre chaque paquet reçu, on attribue à chacune de ces utilisations un **port** spécifique sous la forme d'un entier sur 2 octets. Certains ports sont fixes (un serveur web utilise le port 80 ou le port 443) d'autres sont aléatoires (client web, client de messagerie,...)

6) Commandes réseau utiles

	Linux	Windows
Quelles sont mes adresses MAC et IP ?	<code>ip a</code>	<code>ipconfig /all</code>
Quelle est l'adresse IP de ma passerelle ?	<code>ip r</code>	<code>ipconfig /all</code>
Liste des adresses MAC que mon ordinateur connaît sur le LAN	<code>ip n</code>	<code>arp -a</code>
Tester la connexion entre mon ordinateur et tfontanet.github.io	<code>ping tfontanet.github.io</code>	<code>ping tfontanet.github.io</code>
Liste des routeurs rencontrés pour atteindre le site tfontanet.github.io	<code>tracert tfontanet.github.io</code>	<code>tracert tfontanet.github.io</code>

A faire sur le chromebook :

- Dans les paramètres du chromebook, sélectionnez la connexion wifi en cours et affichez l'adresse MAC et l'IP de votre carte wifi, le masque de sous réseau associé, et l'IP de la passerelle.
- Quelle est la plage d'adresse IP qui est dans le même LAN que votre chromebook ?
- Demandez à vos voisins de classe les IP de leurs cartes wifi. Sont-ils dans le même LAN que votre chromebook ?

A faire sur la machine virtuelle Linux du chromebook :

- Utilisez les commandes ci-dessus pour déterminer l'adresse MAC et l'IP de l'interface réseau de votre machine virtuelle Linux, le masque de sous réseau associé, et l'IP de la passerelle.
- Trouvez-vous les mêmes valeurs que ci-dessus ?
- Quelle est la plage d'adresse IP qui est dans le même LAN que votre machine virtuelle ?
- Vérifiez que l'IP de votre passerelle en fait bien partie.
- Envoyez un ping au site « stmicheldepicpus.fr ». Quelle est l'adresse IP de ce site ?
- Combien de routeurs avez vous traversé pour atteindre ce site ? Est-ce que ce sont les mêmes que vos voisins ?

II) Modèle TCP/IP et protocoles

L'une des forces d'internet est sa souplesse. Cette souplesse a été rendue possible notamment grâce à une grande modularité et un bon découpage des tâches à accomplir. Pour chacune de ces tâches à accomplir, on a défini un certain nombre de règles de communication appelées **protocole** (Par exemple : HTTP, TCP, IP, DHCP, ...). Puis, ces protocoles ont été répartis en couches successives bien identifiées communiquant les unes avec les autres.

1) Un découpage en couches successives

On distingue historiquement deux découpages en couches : le **modèle OSI** et le **modèle TCP/IP**. Nous nous intéresserons principalement au modèle TCP/IP.

OSI	TCP/IP	Tâches	Adresses	Protocoles	Données	Matériel
7-Application	Application			DNS, HTTP, FTP, POP, IMAP, SMTP, CIFS, NFS, SSH	Données Message	
6-Présentation						
5-Session						
4-Transport	Transport	Découpage des gros fichiers en morceaux Gestion des erreurs de transmissions Réassemblage final des gros fichiers	Ports	TCP, UDP	Segment	
3-Réseau	Internet	Routage des données d'une machine à l'autre jusqu'à la bonne destination	IP	IP, DHCP	Datagramme (Paquet)	Routeur
2-Lien	Lien	Communication entre 2 machines directement connectées ou via un switch (qui sont donc sur le même LAN)	MAC	Ethernet	Trame	Switch
1-Physique		Signaux électriques ou lumineux			Bits	Hub

2) Le protocole TCP (Transmission Control Protocol)

- Ce protocole permet à l'ordinateur de départ de créer une connexion avec l'ordinateur destinataire, de vérifier que ce dernier est prêt à recevoir des données et de garantir ensuite un échange fiable et sans pertes.
- Le TCP de l'émetteur découpe les gros envois de données en morceaux appelés segments qu'il numérote. Cela permet d'éviter de devoir tout renvoyer quand un segment est corrompu à l'arrivée et cela permet également de fluidifier les envois (si un petit fichier est envoyé après un très gros, le petit fichier n'est pas obligé d'attendre la fin de l'envoi du très gros).
- Le TCP du récepteur vérifie que chaque segment est bien arrivé. Au besoin, il redemande à l'émetteur les segments manquants ou corrompus, et les réassemble dans le bon ordre.

3) Le protocole IP (Internet Protocol)

- Ce protocole permet d'acheminer les données vers la bonne destination.
- Entre l'émetteur et le récepteur, il y a en général de nombreux **routeurs** connectés les uns aux autres formant comme une toile d'araignée. Le protocole IP va permettre que les données soient envoyées de routeur en routeur vers la bonne destination finale.
- En revanche, IP ne garantit pas l'ordre de réception des paquets, ni même qu'ils ne se perdent pas. Des paquets du même message peuvent emprunter des chemins différents selon l'encombrement du réseau.

4) Envoi d'un message et encapsulation des données

Supposons que je veuille afficher sur mon ordinateur la page d'accueil du site `tfontanet.github.io`. Mon navigateur va formuler une demande qui va ensuite être formatée par mon système d'exploitation pour pouvoir traverser internet. Regardons ci-dessous les différentes étapes du processus :

Départ de mon ordinateur :

Mon navigateur va générer un message respectant le protocole HTTP (ci-dessous en bleu foncé à droite). Puis ce message va traverser les différentes couches du modèle TCP/IP et recevoir à chaque fois un entête supplémentaire. On dit que l'on **encapsule** le message. A la fin la **trame Ethernet** ci-dessous est envoyée sur mon réseau local.

Couche Lien Entête Ethernet	Couche Internet Entête IP	Couche Transport Entête TCP	Couche application Message initial HTTP
Mac src : 40:E5:67:22:AB:81 Mac dest : 67:3F:65:B9:0C:56	IP src : 192.168.0.34 IP dest : 212.27.63.136	TTL : 64 Port src : 52318 Port dest : 80	Msg n°1 Flag GET / HTTP/1.1 Host: tfontanet.github.io

Remarques :

- « IP src » et « Mac src » correspondent aux adresses IP et MAC de mon ordinateur et sont nécessaires pour que le serveur distant puisse me répondre quand il aura reçu le message.
- « IP dest » est l'IP du serveur distant. Si mon ordinateur ne la connaît pas, il doit faire au préalable une **requête DNS** pour savoir à quelle adresse IP correspond le site « `tfontanet.github.io` ».
- Grâce à son masque de sous-réseau, mon ordinateur voit que « IP dest » n'est pas sur le même LAN que lui. Il doit donc envoyer la trame sur sa passerelle. « Mac dest » est donc l'adresse MAC de cette dernière. Si mon ordinateur ne la connaît pas, il doit faire au préalable une **requête ARP** pour savoir quelle est l'adresse MAC de la passerelle.

Traversée d'un switch :

Le switch est un équipement qui ne connaît que les adresses MAC du réseau local. Il a en général 8, 16 ou 24 ports (prises RJ45) et il tient à jour sa **table CAM** dans laquelle il enregistre quelle(s) adresses MAC sont accessibles via quels ports physiques.

Quand la trame arrive sur un switch, celui-ci lit « Mac dest » dans l'entête Ethernet et regarde dans sa table CAM sur quel port il doit envoyer cette trame.

Traversée d'un routeur :

Le routeur est un équipement qui interconnecte plusieurs réseaux locaux grâce à ses interfaces (= cartes réseau). Il met à jour en permanence sa **table de routage** et y enregistre quelles plages d'adresses IP envoyer à quelle interface.

Quand la trame arrive sur un routeur, celui-ci désencapsule l'entête Ethernet qui est obsolète, lit « IP dest » dans l'entête Internet et regarde dans sa table de routage sur quelle interface il doit l'envoyer. Il décrémente également le compteur TTL. Une fois TTL arrivé à 0, le paquet est détruit. Cela permet d'éviter que des paquets perdus n'encombrent inutilement le réseau.

Puis le routeur réencapsule le paquet avec son adresse MAC et celle du routeur de destination.

Arrivée sur le serveur :

L'ordinateur va désencapsuler adresses MAC, IP et ports, puis adresser le message HTTP initial au logiciel serveur web qui écoute le port destination. En réponse, ce logiciel va renvoyer un message beaucoup plus long qui contiendra le code html de la page d'accueil demandée. L'ordinateur va alors réencapsuler ce nouveau message en inversant par rapport à tout à l'heure les ports, IP et MAC sources et destination.

La même chose très bien expliquée en vidéo : https://www.youtube.com/watch?v=_0thnFumSdA

III) Pour aller plus loin

Quelques vidéos complémentaires :

- Récapitulatif de ce qui précède : <https://www.youtube.com/watch?v=dCknqcjcltU>
- Protocole ARP : https://www.youtube.com/watch?v=F3Jn_aqloVQ
- Masques de sous-réseau : https://www.youtube.com/watch?v=ImAtjunA_hI
- Serveur DNS : <https://www.youtube.com/watch?v=qzWdzAvfBoo>
- Serveur DHCP : <https://www.youtube.com/watch?v=44ONmO8qDw8>
- Adresses IP privées et translation d'adresses NAT et PAT : <https://www.youtube.com/watch?v=jq3SLuhIyPI>

Et un cours remarquable de l'EPFL qui reprend ce qui précède :

- https://www.youtube.com/watch?v=U6Uqf5xsaSI&list=PLOapGKeH_KhFkfZDf5B-AKXmNIXEPpn4a

ALGORITHMES CLASSIQUES

I) Complexité d'un algorithme

Définition :

On appelle **complexité** d'un algorithme la façon dont le nombre d'*actions élémentaires* effectuées par cet algorithme évolue en fonction de la taille des données traitées. (On entend par action élémentaire : les affectations, comparaisons,...)

Ex : recherche d'un minimum dans une liste

```
def minimum(liste):  
    min = liste[0]  
    for item in liste:  
        if item < min:  
            min = item  
    return min
```

Pour trouver le minimum, l'action élémentaire répétée un grand nombre de fois est la comparaison `item < min`. La fonction a fait autant de comparaisons qu'il y a d'éléments dans la liste. On dit que cet algorithme a une complexité **linéaire** : Si la liste est 10 fois plus grande, l'algorithme mettra environ 10 fois plus de temps à s'exécuter.

Vérification expérimentale :

En multipliant ci-dessous `taille_liste` par 10, 100 ou 1000, comment évolue la durée du script ?

```
import random, time  
  
taille_liste = 10**5  
liste = [random.randint(0,100) for i in range(taille_liste)]  
  
début = time.time()  
a = minimum(liste)  
fin = time.time()  
  
print(f"n = {taille_liste}, durée = {fin - début} secondes")
```

Remarques :

- On distingue parfois la complexité « en temps » et la complexité « en mémoire ». Nous ne considérerons cette année que la complexité en temps.
- Quand on cherche à déterminer la complexité d'un algorithme, on ne cherche pas à compter précisément le nombre d'actions élémentaires effectuées : un simple ordre de grandeur suffit.
- La complexité d'un algorithme peut être notamment :
 - constante (ne dépend pas de la taille n des données traitées)
 - linéaire (proportionnelle à n , par exemple lors d'une boucle simple)
 - quadratique (proportionnelle à n^2 , par exemple s'il y a 2 boucles imbriquées)
- Une vidéo remarquable sur la question : <https://www.youtube.com/watch?v=AgtOCNCejQ8>

II) Recherche d'une valeur dans une liste

1) Cas d'une liste non triée : Recherche par « balayage »

On veut écrire une fonction `recherche_balayage` déterminant si une valeur est dans une liste ou non.

Bien sûr, nous nous interdisons d'utiliser l'expression « `valeur in liste` » qui ferait le travail à notre place ;-)

Principe :

Cette fonction va balayer la liste de gauche à droite à la recherche de la valeur demandée et s'arrêter dès que cette dernière est trouvée. A la fin du balayage, si la valeur demandée n'a pas été trouvée, alors cette valeur n'est pas dans la liste.

Code (à compléter) :

```
def recherche_balayage(valeur, liste):  
    for   
        if valeur == item:  
            return   
    return   
  
liste = [3, 9, 5, 17, 12, 0, -4]  
assert recherche_balayage(9, liste)  
assert recherche_balayage(-4, liste)  
assert not recherche_balayage(10, liste)
```

Complexité :

Quelle est la complexité de cet algorithme « dans le pire des cas » ?

2) Cas d'une liste triée : Recherche par dichotomie

Si la liste est triée, on peut résoudre le même problème de façon plus astucieuse.

Exemple : Le nombre 15 appartient-il à la liste triée ci-dessous ?

item	2	3	5	5	9	11	13	14	17	19	20
indice	0	1	2	3	4	5	6	7	8	9	10

Principe :

On va diviser par deux la plage de recherche à chaque étape en comparant 15 avec le terme qui est au milieu de cette plage.

	Étapes				
	1	2	3	4	5
Indice du début de plage : a =	0				
Indice de fin : b =	10				
Indice du milieu : i =	5				
On compare 15 avec :	11				

Code (à compléter) :

```
def recherche_dichotomie(valeur, liste):  
    a =   
    b =   
    while a <= b:  
        i = (a+b)//2  
        if valeur == liste[i]:  
              
        elif valeur > liste[i]:  
              
        else:  
              
    return False  
  
liste = [2,3,5,5,9,11,13,14,17,19,20]  
assert not recherche_dichotomie(15, liste)  
assert recherche_dichotomie(11, liste)  
assert recherche_dichotomie(20, liste)
```

Complexité :

La complexité de cet algorithme est meilleure que celle de l'algorithme par balayage. En effet, même s'il y a plus d'actions élémentaires par passage dans la boucle, il y a beaucoup beaucoup moins de passages dans la boucle !

Observons le tableau ci-dessous :

D'une colonne à la suivante, on a multiplié par 10 la taille de la liste de départ.

D'une ligne à la suivante, on a divisé par deux la taille de la liste comme le fait la dichotomie.

100	1000	10000	100000	1000000	10000000	100000000	1000000000	10000000000	100000000000
50	500	5000	50000	500000	5000000	50000000	500000000	5000000000	50000000000
25	250	2500	25000	250000	2500000	25000000	250000000	2500000000	25000000000
12	125	1250	12500	125000	1250000	12500000	125000000	1250000000	12500000000
6	62	625	6250	62500	625000	6250000	62500000	625000000	6250000000
3	31	312	3125	31250	312500	3125000	31250000	312500000	3125000000
1	15	156	1562	15625	156250	1562500	15625000	156250000	1562500000
	7	78	781	7812	78125	781250	7812500	78125000	781250000
	3	39	390	3906	39062	390625	3906250	39062500	390625000
	1	19	195	1953	19531	195312	1953125	19531250	195312500
		9	97	976	9765	97656	976562	9765625	97656250
		4	48	488	4882	48828	488281	4882812	48828125
		2	24	244	2441	24414	244140	2441406	24414062
		1	12	122	1220	12207	122070	1220703	12207031
			6	61	610	6103	61035	610351	6103515
			3	30	305	3051	30517	305175	3051757
			1	15	152	1525	15258	152587	1525878
				7	76	762	7629	76293	762939
				3	38	381	3814	38146	381469
				1	19	190	1907	19073	190734
					9	95	953	9536	95367
					4	47	476	4768	47683
					2	23	238	2384	23841
					1	11	119	1192	11920
						5	59	596	5960
						2	29	298	2980
						1	14	149	1490
							7	74	745
							3	37	372
							1	18	186
								9	93
								4	46
								2	23
								1	11
									5
									2
									1
6	9	13	16	19	23	26	29	33	36

En dernière ligne, on a précisé le nombre d'étapes qui ont été nécessaires pour atteindre 1.

On remarque que quand on multiplie par 10 la taille de la liste, il faut au nombre d'étapes.

Trouvez-vous sur internet le nom de la fonction mathématique célèbre qui permet de faire ce genre de choses ?

On retiendra que la complexité de la recherche par dichotomie est

Terminaison :

Dans cet algorithme, nous avons utilisé une boucle **while**. Or il y a toujours un risque que ce genre de boucle soit infinie ! On appelle **prouver la terminaison** d'un algorithme le fait de prouver que l'exécution de l'algorithme va s'arrêter quelles que soient les entrées.

Pour cela, nous allons montrer que $b - a$ est un **variant de boucle**, c'est-à-dire un entier positif qui décroît strictement à chaque itération.

Ici l'algorithme s'arrête quand le variant de boucle $b - a$ devient strictement négatif.

A chaque itération, on entre dans la boucle uniquement si $a \leq b$,

puis, après modification de i , on obtient : $a \leq i \leq b$.

En fin de boucle :

- soit $a = i + 1$.

Donc a est incrémenté d'au moins 1 et le variant de boucle est décrémenté d'au moins 1

- soit $b = i - 1$.

Donc b est décrémenté d'au moins 1 et le variant de boucle est aussi décrémenté d'au moins 1

Bilan, à chaque itération le variant de boucle décroît d'au moins 1. Il finira donc bien par devenir négatif.

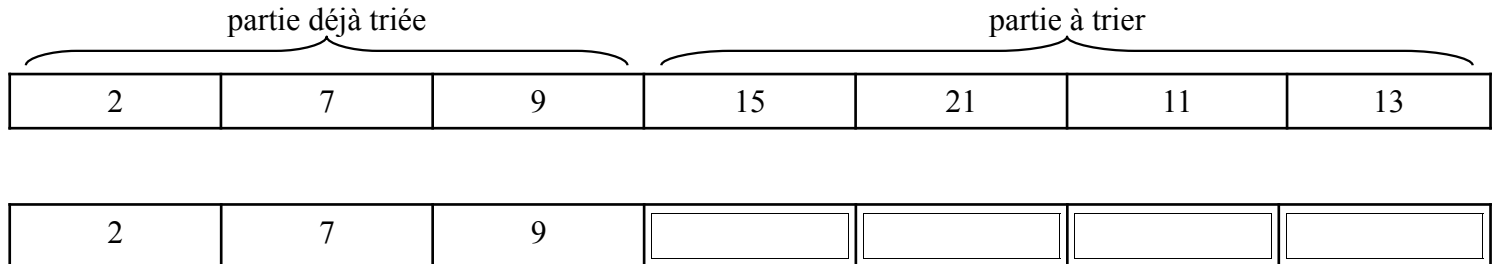
III) Algorithmes de Tri

1) Tri par sélection

Principe :

A chaque étape d'un **tri par sélection**, la partie gauche de la liste est déjà triée. On parcourt alors la partie droite à la recherche du minimum que l'on échange avec le premier terme de cette partie droite.

Exemple d'étape de tri :



La même chose avec une animation visuelle : <https://visualgo.net/en/sorting>

Code (à compléter) :

```
def tri_selection(liste):  
    # Dans la partie restant à trier :  
    #   i désigne l'indice du premier terme  
    #   imin désigne l'indice du minimum  
  
    for i in range(  
        imin =   
        for j in range(  
            if liste[j] < liste[imin]:  
                imin =   
  
        # On permute le minimum de la partie à trier avec le premier terme  
        liste[i], liste[imin] =   
  
from random import randint  
for i in range(10):  
    l = [randint(-10, 10) for i in range(10)]  
    tri_selection(l)  
    assert l == sorted(l)
```

Complexité :

Si n est la taille de la liste, le nombre de fois où on exécute la boucle intérieure est :

$$(n - 1) + \dots + 4 + 3 + 2 + 1 =$$

La complexité est donc

2) Tri par insertion

Principe :

A chaque étape d'un **tri par insertion**, la partie gauche de la liste est déjà triée et on cherche à y insérer au bon endroit le premier nombre de la partie droite. On sauvegarde donc ce nombre à insérer et on le compare successivement à chacun des nombres de la partie triée en commençant par les plus grands. Si le nombre de la partie triée est plus grand, on le décale d'une case à droite, sinon on écrase le nombre de droite par le nombre à insérer.

Exemple d'étape de tri :

partie triée				partie à insérer		
2	7	9	9	5	4	11
2					4	11

Code (à compléter) :

```
def tri_insertion(liste):
    # i désigne l'indice du terme à insérer (=le plus à gauche de la partie non triée)
    # j désigne l'indice du terme de la partie triée auquel on le compare

    for i in :

        terme_à_insérer = 

        j = 

        while j >= 0 and :

            liste[j+1] = 

            j -= 1

        liste[j+1] = 

from random import randint
for i in range(10):
    l = [randint(-10, 10) for i in range(10)]
    tri_insertion(l)
    assert l == sorted(l)
```

Complexité :

Si n est la taille de la liste, le nombre de fois où on exécute la boucle while est dans le pire des cas :

$1 + 2 + 3 + 4 + \dots + (n - 1) =$ La complexité est donc

Terminaison :

Il y a une boucle **while**. Choisissons j comme variant de boucle (première condition du **while**). A l'intérieur de la boucle, j est décrémenté donc ce variant finira donc bien par devenir négatif même si la deuxième condition du **while** reste toujours vraie.

IV) Algorithme des *k* plus proches voisins

L'exemple des Iris de Fisher

L'algorithme des **k plus proches voisins** appelé aussi **knn** (k nearest neighbors) est un algorithme de classification très simple qui fait partie de la famille des algorithmes d'apprentissage (machine learning).

Dans tout algorithme d'apprentissage, il faut un **jeu de données**. Nous utiliserons ci-dessous l'exemple très célèbre des « iris de Fisher ». Dans ce jeu de données, un botaniste a mesuré les pétales de 150 iris de trois espèces différentes : Les iris setosa, les iris versicolor et les iris virginica.

Iris setosa



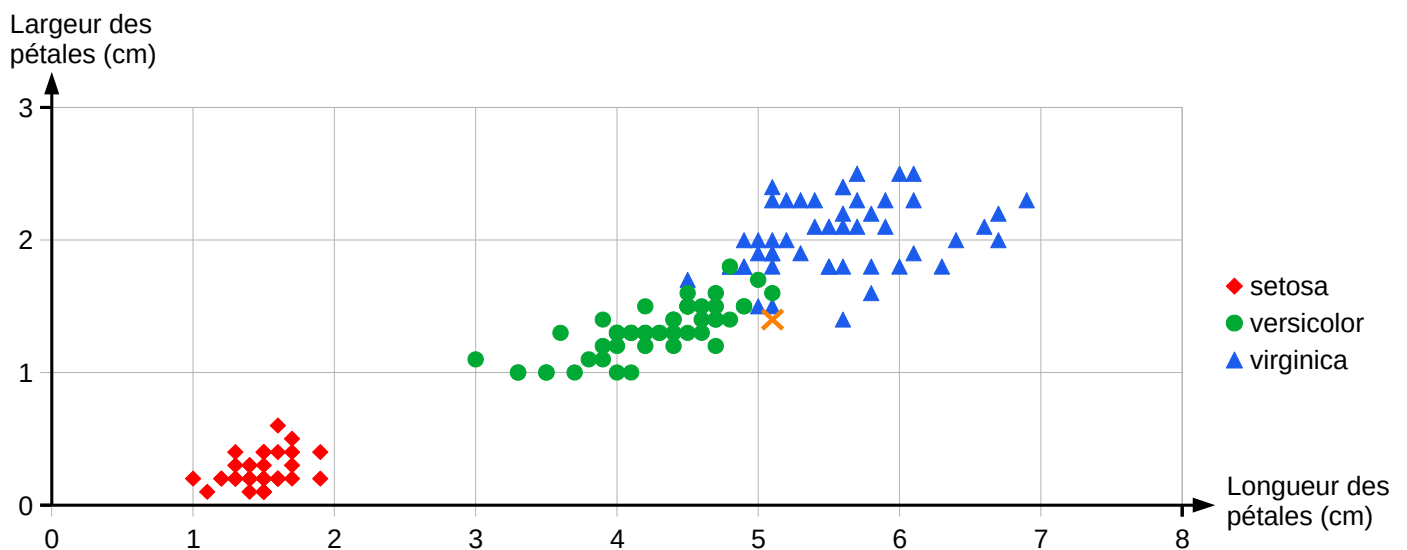
Iris versicolor



Iris virginica



On a représenté les données mesurées par le botaniste dans la figure ci-dessous :



Principe de l'algorithme :

On vient de trouver un iris dans un champ et on cherche son espèce en s'appuyant sur notre jeu de données :

- On choisit une valeur de *k* appropriée (*k* = le nombre de voisins que l'on va considérer).
- On calcule les distances entre l'iris que l'on a trouvé et chaque iris du jeu de données.
- On identifie l'espèce majoritaire parmi les *k* plus proches voisins de l'iris trouvé.

Application :

Dimensions des pétales	Espèce si <i>k</i> = 2	Espèce si <i>k</i> = 5
1,5 cm × 0,3 cm		
4 cm × 1 cm		
7 cm × 2 cm		
iris symbolisé par une croix orange		

Remarques :

- Pour calculer les distances, nous avons utilisé ici la distance euclidienne : $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.
Rappel, pour visualiser une distance euclidienne sur un graphique, le repère doit être orthonormé !
- Selon les situations, on peut aussi utiliser d'autres types de distances : Manhattan, Hamming,...
- Le choix de k (nombre de voisins retenus) se fait souvent empiriquement en testant plusieurs valeurs.

Exercices :

1) Écrire un script Python qui :

- demande à l'utilisateur la longueur et la largeur d'un nouveau pétale d'iris.
- crée la liste des iris du jeu de données « iris.csv ». Chaque iris sera représenté par une sous-liste de la forme : [distance euclidienne entre le pétale de l'iris et le nouveau pétale, longueur du pétale de l'iris, largeur du pétale de l'iris, espèce de l'iris].
- trie cette liste de liste selon les distances entre le nouvel iris et les iris du jeu de données.
- Affiche les espèces des 5 plus proches voisins du nouvel iris.

2) Utiliser l'algorithme des k plus proches voisins pour aider le choixpeau magique de Poudlard à déterminer la maison d'Hermione.

- Jeu de données : « choixpeau-magique.csv ».
- Hermione a pour caractéristiques : Courage=8, Loyauté=6, Sagesse=6, Malice=6.
- k = 7
- Distance utilisée : Manhattan (faites une recherche sur le web pour savoir de quoi il s'agit ;-)

3) Utiliser l'algorithme des k plus proches voisins pour décider si un texte est écrit en français ou en anglais.

On s'appuiera pour cela sur le fait que la fréquence des « h » et celle des « y » sont assez différentes dans les deux langues et on créera un jeu de données en analysant ces fréquences dans quelques textes des deux langues.

V) Algorithmes gloutons

1) Problèmes d'optimisation

Un **problème d'optimisation** est un problème dont la difficulté n'est pas de trouver des solutions mais plutôt de trouver la « meilleure » solution.

Exemples :

- Rendre la monnaie avec un minimum de pièces.
- Mettre le plus de livres possible dans un carton.
- Trouver la route la plus rapide entre deux villes.

2) Algorithmes glouton

Pour résoudre un problème d'optimisation, on peut chercher toutes les solutions, puis définir ensuite la meilleure. Malheureusement, dans certains cas, trouver toutes les solutions peut être très long, même pour un ordinateur !

On peut alors essayer d'utiliser un **algorithme glouton** qui va procéder étape par étape, en faisant à chaque étape le meilleur choix possible à ce moment là.

Remarques :

- Le gros intérêt des algorithmes gloutons est qu'ils sont beaucoup plus rapides que la recherche de toutes les solutions.
- En revanche, on verra ci-dessous qu'un algorithme glouton ne trouve pas toujours la meilleure des solutions et même parfois ne trouve pas de solution du tout alors qu'il en existe !

3) Le rendu de monnaie

Commençons par un grand classique : Comment rendre la monnaie avec un nombre minimal de pièces.

a) Système de monnaie habituel

On doit rendre 162 euros en utilisant les pièces suivantes en nombre illimité : 100 ; 50 ; 20 ; 10 ; 5 ; 2 ; 1 €.

	Somme restant à rendre	Pièces rendues
Étape 0	162	aucune
Étape 1	62	100
Étape 2	12	100 ; 50
Étape 3	2	100 ; 50 ; 10
Étape 4	0	100 ; 50 ; 10 ; 2

Le principe est simple : à chaque étape, on rend la pièce la plus grosse possible et notre système de monnaie est fait de telle sorte que cet algorithme donnera toujours la meilleure solution.

b) Système de monnaie exotique

On doit rendre 162 euros en utilisant maintenant les pièces suivantes en nombre illimité : 50 ; 40 ; 3 ; 2 €.

En appliquant le même algorithme que ci-dessus, combien de pièces devra-t-on rendre ?

Peut-on trouver une solution permettant de ne rendre que 5 pièces ?

Avec ce nouveau système de monnaie, nous voyons que l'algorithme glouton permet de trouver une solution pour 162 euros mais celle-ci n'est pas optimale.

De même cet algorithme n'aurait pas permis de rendre 160 euros alors que $160 = 40 + 40 + 40 + 40$.

c) Code (à compléter)

```
def rendu_glouton(somme_à_rendre, monnaie):  
    """  
    Paramètres :  
        somme_à_rendre : int  
        monnaie : tuple ou list trié dans l'ordre décroissant  
    Valeur renvoyée :  
        liste_pièces_à_rendre : list  
        (renvoie une liste vide dans le cas où l'algorithme ne trouve pas de solution)  
    """  
    liste_pièces_à_rendre = []  
  
    if somme_à_rendre == 0:  
        return liste_pièces_à_rendre  
  
    assert rendu_glouton(162, (100, 50, 20, 10, 5, 2, 1)) == [100, 50, 10, 2]  
    assert rendu_glouton(162, (50, 20, 10, 5, 2, 1)) == [50, 50, 50, 10, 2]  
    assert rendu_glouton(162, (50, 40, 3, 2)) == [50, 50, 50, 3, 3, 3, 3]  
    assert rendu_glouton(160, (50, 40, 3, 2)) == []
```

d) Conclusion

Si on veut dénombrer toutes les façons de rendre la monnaie pour choisir ensuite la meilleure, on trouve déjà plus de 30 000 possibilités dans le cas a) !

Certes notre algorithme glouton n'est optimal que si le système de monnaie est bien conçu mais il a quand même l'avantage de nous conduire rapidement à une solution satisfaisante !

4) Le problème du sac à dos

Autre grand classique !

Vos parents vous envoient acheter ce qu'il faut pour l'apéritif. Vous avez le droit aux articles ci-dessous avec deux contraintes : Vous n'avez pas le droit de choisir deux fois le même article et tout doit tenir dans un sac ridicule qui contient au maximum 600 g !

Qu'allez vous choisir pour maximiser le prix du contenu du sac à dos sans dépasser la masse autorisée ?

Article	Chips bio	Pistaches	Curly	Noix de pécan	Saucisson
Masse (g)	100	300	100	100	300
Valeur (€)	2,4	4,55	1,30	3,80	6,25
Valeur/Masse	0,024	0,015	0,013	0,038	0,021

a) Premier essai :

Vous décidez de choisir en premier les articles les plus chers. Est-ce un algorithme glouton ?

Quels articles prenez-vous ? Quelle est la valeur du sac ?

b) Deuxième essai :

Vous décidez de choisir en premier les articles ayant le ratio « valeur/masse » le plus avantageux.

Est-ce un algorithme glouton ? Quels articles prenez-vous ?

Quelle est la valeur du sac ?

Code (à compléter)

```
def sac_à_dos_glouton(masse_max, liste_articles):
    articles = sorted(liste_articles, reverse = True)
    contenu_sac_à_dos = 
    valeur_totale = 
    masse_totale = 
    for article in articles:
        
        
        
        
    return contenu_sac_à_dos, valeur_totale, masse_totale

liste_articles = [[0.024, 2.4, 100, "Chips bio"],
                  [0.015, 4.55, 300, "Pistaches"],
                  [0.013, 1.3, 100, "Curly"],
                  [0.038, 3.8, 100, "Noix de pécan"],
                  [0.021, 6.25, 300, "Saucisson"]]
assert sac_à_dos_glouton(600, liste_articles) == (['Noix de pécan', 'Chips bio',
'Saucisson', 'Curly'], 13.75, 600)
```

c) Conclusion :

Si on fait abstraction de la contrainte des 600 g, combien y a-t-il de façon de remplir le sac avec 5 articles ?

Même question avec 15 articles ?

On voit que l'étude de toutes les solutions est de complexité factorielle alors que la complexité de l'algorithme glouton (au tri de la liste des articles près) est linéaire.

Pour approfondir la question : <https://interstices.info/le-probleme-du-sac-a-dos/>

INTERACTIONS CLIENT-SERVEUR SUR LE WEB

Après avoir étudié comment écrire des pages web, intéressons nous maintenant aux mécanismes qui permettent à un internaute de les consulter.

I) Le modèle client-serveur

1) Introduction

Sur internet, la plupart des échanges de données fonctionnent sur le modèle **client-serveur**. Un **client** (par exemple l'ordinateur d'un internaute) fait une requête à un **serveur** (souvent situé dans un data center), puis ce dernier lui répond en lui envoyant les données demandées (par exemple un mail, une page web, une vidéo,...).

Selon le contexte, les termes **client** et **serveur** désignent soit des ordinateurs, soit les logiciels tournant sur ces ordinateurs et permettant la communication.



Exemple de serveur utilisé dans un data center

2) Dans le cas du web :

	Ordinateur	Logiciels	Langages
Client	Ordinateurs des internautes	Navigateurs des internautes : Chrome, Firefox, Edge, Safari, ...	HTML, CSS, Javascript
Serveur	Ordinateurs spécialisés allumés 24h/24	Serveurs web : Apache, Nginx, IIS,...	Php, Python, Java,...

3) Pages web « statiques » et pages web « dynamiques »

On a coutume de distinguer les pages web **statiques** (qui sont enregistrées « telles quelles » sur les disques durs des serveurs Web) et les pages web **dynamiques** (dont le code HTML-CSS-JavaScript est généré par le serveur au moment où l'internaute veut les consulter).

Par exemple, ce site Web est statique alors que le site de Météo France est dynamique car la météo change tout le temps et n'est pas la même selon les lieux !

Dans le cas d'une page web dynamique, le serveur va « calculer » le code HTML-CSS-JavaScript à envoyer au client à l'aide de scripts écrits dans langages de programmation comme le Php, Python ou Java. Ces langages serveurs interagissent souvent avec une base de donnée.

Remarque : Un code Javascript étant exécuté sur le navigateur du client, il ne peut agir que sur la page web en cours, alors qu'un code Php est exécuté sur le serveur et peut modifier ce qui est stocké sur le serveur.

II) Le protocole HTTP

Un navigateur et un serveur web communiquent en utilisant le protocole HTTP (ou HTTPS) qui est un protocole de la couche .

1) Requêtes HTTP

Supposons que l'on veuille consulter le site <https://tfontanet.github.io/>.

- 1) L'internaute tape l'adresse du site dans la barre d'adresse du navigateur ou clique sur un lien.
- 2) Le navigateur rédige une **requête HTTP** pour demander la page d'accueil du site au serveur qui héberge le domaine tfontanet.free.fr.
- 3) Le serveur reçoit la requête et y répond en envoyant le source html de la page web demandée.
- 4) Le navigateur lit le source html et identifie les ressources supplémentaires dont il va avoir besoin pour afficher la page (images, feuilles css,...), puis il envoie au serveur des **requêtes** supplémentaires pour obtenir ces ressources.
- 5) Le navigateur reçoit ces ressources au fur et à mesure et affiche la page web.

Exemple de requête envoyée par le navigateur :

```
GET / HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate
Connection: keep-alive
Host: tfontanet.github.io
User-Agent: Chrome/114.0.0.0 (X11; Linux x86_64)
```

Cette requête commence par : **GET / HTTP/1.1**

- **GET** désigne la « méthode HTTP » à utiliser (voir ci-dessous).
- **/** précise le nom du fichier demandé au serveur (Ici, le slash désigne la page d'accueil du site).
- **HTTP/1.1** correspond à la version du protocole HTTP utilisée.

A cette requête, le serveur va répondre par un message en deux parties avec **entête** puis **données** :

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Connection: close
Content-Length: 2695
Content-Type: text/html
Date: Thu, 27 Jul 2023 14:25:43 GMT
ETag: "84c176-a87-64c0b5ec"
Last-Modified: Wed, 26 Jul 2023 05:58:04 GMT
Server: Apache/ProXad [Jan 23 2019 20:05:46]

<!doctype html>
<html lang="fr">
  <head>
    <title>Cours, devoirs et activités mathématiques à télécharger</title>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" type="text/css" href="style.css">
  </head>
  <body>
    <div id="marges">
      .....
    </div>
  </body>
</html>
```

2) HTTP ou HTTPS ?

Avec le protocole HTTP, les échanges de données se font « en clair ». Si quelqu'un place un équipement pour intercepter le flux de données entre un internaute et un serveur, il aura accès à toutes les données échangées.

Avec le protocole HTTPS, les échanges de données sont chiffrés. Une personne malintentionnée pourra toujours intercepter le flux de données entre un internaute et un serveur mais ce flux sera inexploitable pour elle car elle ne dispose pas de la clé de déchiffrement.

Tant qu'un internaute consulte des sites publics sur lesquels il ne transmet aucune information personnelle (identifiants,...), le protocole HTTP est suffisant. En revanche, s'il doit par exemple s'identifier sur un site, il est prudent de vérifier que la connexion est bien en HTTPS pour éviter que le mot de passe ne transite en clair sur le réseau.

3) Code de réponse du serveur

Dans la première ligne de la réponse du serveur, il y a toujours un code HTTP.
Citons ici quelques codes à connaître :

Code	Signification
200	
304	
404	

3) Méthodes GET et POST

Une requête HTTP commence par un mot clé qui précise le type d'action attendu. On parle de « méthode HTTP ».

Lors de l'envoi d'un formulaire, il faut choisir entre **GET** et **POST** :

- La méthode GET va insérer les valeurs des paramètres dans l'URL (peu discret, mais pratique si on veut que le navigateur mémorise nos réponses).

Ex avec le formulaire précédent :

```
GET /1nsi/1nsi-client-serveur-reponse.php?nom=Thibaud&nsi=Yes HTTP/1.1
Host: fdcsjh.free.fr
```

- La méthode POST va insérer les valeurs des paramètres dans le corps de la requête (plus discret, et même indispensable s'il y a un volume important de données à transmettre).

Même requête que ci-dessus mais avec la méthode POST :

```
POST /1nsi/1nsi-client-serveur-reponse.php HTTP/1.1
Host: fdcsjh.free.fr
Content-Type: application/x-www-form-urlencoded
Content-Length: 16

nom=Thibaud&nsi=Yes
```

4) Exemple de code Php exécuté côté serveur

Voici le code de la page « 1nsi-client-serveur-reponse.php » :

```
<?php
$methode = "Inconnue";
$nom = "Anonyme";
$nsi = "et vous auriez mieux fait de choisir NSI ;-(";
if (isset($_GET['nom'])) {
    $methode = "GET";
    $nom = $_GET['nom'];
    if ($_GET['nsi'] == "Yes") {$nsi = "et vous avez bien fait de choisir NSI ;-");}
} elseif (isset($_POST['nom'])) {
    $methode = "POST";
    $nom = $_POST['nom'];
    if ($_POST['nsi'] == "Yes") {$nsi = "et vous avez bien fait de choisir NSI ;-");}
}
?>
<!doctype html>
<html lang="fr">
    <meta charset="utf-8">
    <style>
        body {background-color: #e8e8e8; font-family: sans-serif}
        div {margin: auto; max-width: 400px; background-color: #ffffff; padding: 30px}
        fieldset {background-color: #f8f8f8; padding: 15px; border: 3px solid #d4d4d4; border-radius: 10px}
    </style>
</head>
<body>
    <br><br>
    <div>
        <fieldset>
            <legend>Bonjour <?php echo $nom ?></legend>
            <p>Vous avez utilisé la méthode : <?php echo $methode ?></p>
            <p><?php echo "Il est ".date("H:i:s").", votre IP est ".$_SERVER['REMOTE_ADDR']; ?></p>
            <p><?php echo $nsi ?></p>
        </fieldset>
    </div><br><br>
</body>
</html>
```

TRAITEMENT DE DONNÉES EN TABLES

I) Qu'est-ce qu'une « table » ?

Ex : Les personnages de la saga starwars (d'après www.kaggle.com/datasets/jsphvg/star-wars)

id	char	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	planet	species
1		Luke Skywalker	172	77	blond	fair	blue	19BBY	male	59	Human
2		C-3PO	167	75		gold	yellow	112BBY		59	Droid
3		R2-D2	96	32		white, blue	red	33BBY		7	Droid
4		Darth Vader	202	136	none	white	yellow	41.9BBY	male	59	Human
5		Leia Organa	150	49	brown	light	brown	19BBY	female	1	Human
6		Anakin Skywalker	188	84	blond	fair	blue	41.9BBY	male	59	Human
7		Obi-Wan Kenobi	182	77	auburn, white	fair	blue-gray	57BBY	male	19	Human

Remarques :

- En informatique, les données sont souvent organisées en grands tableaux appelés « **tables** ».
- Chaque colonne d'une table correspond à une des informations recueillies. Ces informations sont appelées « **descripteurs** » ou « **attributs** ».
- Chaque ligne d'une table est appelée « **enregistrement** ».
- Dans l'exemple ci-dessus, chaque personnage est unique il y a un unique enregistrement par personnage. Or plusieurs personnages peuvent avoir la même taille, même couleur de cheveux ou date de naissance. Il est donc important qu'au moins une des colonnes n'ait pas de « **doublons** » de façon à pouvoir distinguer de façon unique les personnages. Le champ « name » ne peut pas forcément tenir ce rôle car même si nous ne connaissons qu'un seul Luke Skywalker, peut-être y en a-t-il plusieurs dans la galaxie ? Pour résoudre cette difficulté, on ajoute souvent une colonne (nommée ici « id_char ») associant à chaque enregistrement un « **identifiant unique** ».

II) Les fichiers « CSV »

Parmi les formats de fichiers permettant de stocker un tableau de données, le format CSV (Comma-Separated Values), est un format texte très répandu.

Exemple : Les personnages de la saga starwars au format csv

```
id_char,name,height,mass,hair_color,skin_color,eye_color,birth_year,gender,planet,species
1,Luke Skywalker,172,77,blond,fair,blue,19BBY,male,59,Human
2,C-3PO,167,75,,gold,yellow,112BBY,,59,Droid
3,R2-D2,96,32,,,"white, blue",red,33BBY,,7,Droid
4,Darth Vader,202,136,none,white,yellow,41.9BBY,male,59,Human
5,Leia Organa,150,49,brown,light,brown,19BBY,female,1,Human
6,Anakin Skywalker,188,84,blond,fair,blue,41.9BBY,male,59,Human
7,Obi-Wan Kenobi,182,77,"auburn, white",fair,blue-gray,57BBY,male,19,Human
```

- Chaque ligne du texte correspond à une ligne du tableau précédent et des virgules permettent de séparer le contenu des colonnes.
- La première ligne permet en général de préciser les noms des descripteurs.
- Si une cellule du tableau est vide, alors il y a deux virgules à la suite.
- Si une cellule contient une virgule, alors on « protège » le contenu de cette cellule en l'entourant de guillemets pour que cette virgule ne soit pas interprétée comme un séparateur entre deux colonnes.

Remarques :

La syntaxe des fichiers csv n'est pas complètement standardisée. Quand on ouvre un fichier csv avec un tableur, il est donc prudent de faire quelques vérifications :

- Est-ce que l'encodage des caractères a été reconnu : les accents et caractères spéciaux sont-ils bien retranscrits ?
- Quel est le séparateur entre les colonnes ? Normalement c'est la virgule, mais dans les documents francophones, on choisit souvent le point-virgule car la virgule est déjà utilisée comme séparateur décimal.
- Quel est le séparateur décimal : point ou virgule ? Les nombres décimaux sont-ils bien reconnus ?

III) Avec un tableur

Quand les traitements que l'on souhaite faire sur les données sont simples, un tableur suffit souvent (Microsoft Excel, Libre Office Calc, Google Sheet,...)

Même si ces tableurs ont leurs propres formats de fichiers (.xlsx, .ods), ils savent aussi lire les fichiers csv.

Remarque : Excel a souvent du mal à formater automatiquement les fichiers csv séparés par des virgules. Pour forcer le formatage manuel, choisir le menu « Données », puis « A partir du texte » pour afficher l'assistant d'importation.

A faire :

- 1) Ouvrir le fichier « characters.csv » avec un tableur. Vérifier qu'il est lu correctement : colonnes séparées au bon endroit, encodage.
- 2) Combien y a-t-il d'enregistrements en tout ?
- 3) En vous aidant des possibilités du tableur (tri, recherche, fonction =nb.si(),...), répondre aux questions suivantes :
 - Combien y a-t-il de personnages humains ?
 - Combien y a-t-il de droïdes aux yeux rouges ?
 - Combien y a-t-il de personnages de plus de 2 mètres ?
 - Créer dans un nouvel onglet une feuille de calcul contenant le nom et la couleur de peau des personnages dont la peau contient du marron. (On pourra utiliser =nb.si avec comme critère de recherche "*brown*")
 - Trier la feuille de calcul initiale par taille décroissante.
 - Retrier la feuille de calcul initiale par ordre alphabétique des noms.
 - Créer un nouvel onglet dans lequel on copiera le contenu de planet.csv, puis ajouter à l'onglet contenant characters.csv une colonne avec les noms des planètes d'origine. (On pourra utiliser =recherchev)

IV) Avec Python

Les fichiers csv étant des fichiers textes, ils sont faciles à lire avec Python.
Testons successivement les 3 méthodes ci-dessous pour lire un fichier csv :

1) Méthode 1 : Lecture du fichier csv en Python natif (sans utiliser le module csv)

```
with open("characters.csv") as fichiercsv:
    liste_de_lignes = fichiercsv.readlines()
liste_de_listes = []
for ligne in liste_de_lignes:
    liste_de_listes.append(ligne.strip().split(","))
```

Questions :

- A quoi sert le `.strip()` ?
- A quoi sert le `.split(",")` ?
- Que contient `liste_de_listes[0]` ?
- `liste_de_listes[3]` contient quel enregistrement du fichier ?
- Comment afficher juste la taille du droïde R2-D2 ?
- Est-ce que sa couleur de peau a été bien interprétée ?

2) Méthode 2 : Lecture du fichier csv avec la méthode "reader" du module csv

```
import csv
with open("characters.csv", newline='') as fichiercsv:
    liste_de_listes = list(csv.reader(fichiercsv))
```

Questions :

- De quel type est l'objet `csv.reader` ?
- A quoi sert le `list()` ?
- Que contient `liste_de_listes[0]` ?
- `liste_de_listes[3]` contient quel enregistrement du fichier ?
- Comment afficher juste la taille du droïde R2-D2 ?
- Est-ce que sa couleur de peau a été bien interprétée ?

3) Méthode 3 : Lecture du fichier csv avec la méthode "DictReader" du module csv

```
import csv
with open("characters.csv", newline='') as fichiercsv:
    liste_de_dictionnaires = list(csv.DictReader(fichiercsv))
```

Questions :

- Que contient `liste_de_listes[0]` ?
- `liste_de_listes[3]` contient quel enregistrement du fichier ?
- Comment afficher juste la taille du droïde R2-D2 ?
- Laquelle des 3 méthodes ci-dessus vous paraît préférable ?